



دانشگاه زنجان

دانشکده مهندسی

گروه کامپیوتر

پایان نامه کارشناسی

گرایش نرم افزار

عنوان: سامانه مدیریت و تحلیل هوشمند فرآیند اکوکار دیوگرافی با بهره‌گیری از پردازش
تصویر، سیستم استنتاج فازی و سخت‌افزار مانیتورینگ ضربان

دانشجویان:

نرگس آهنگر ۴۰۱۴۶۳۱۲۰

محمد رضا مهری ۴۰۱۴۶۳۱۷۶

استاد راهنما: دکتر داود محمدپور زنجان

تاریخ دفاعیه: تیر ۱۴۰۵

بِسْمِ اللَّهِ الرَّحْمَنِ الرَّحِيمِ

فرم نمره نهایی پروژه کارشناسی - گروه مهندسی کامپیوتر دانشگاه زنجان

نام و نام خانوادگی دانشجویان	نرگس آهنگر ، محمدرضا مهری
شماره دانشجویی	۴۰۱۴۶۳۱۷۶ ، ۴۰۱۴۶۳۱۲۰
عنوان پروژه	سامانه مدیریت و تحلیل هوشمند فرآیند اکوکاردیوگرافی با بهره‌گیری از پردازش تصویر، سیستم استنتاج فازی و سخت‌افزار مانتورینگ ضربان
نمره نهایی به عدد	
نمره نهایی به حروف	
تاریخ دفاع	
نام استاد راهنما	دکتر داود محمدپور
نام مدیرگروه	دکتر امید عباس زاده

فهرست مطالب

چکیده.....	۱
فصل اول: مقدمه و معرفی پروژه.....	۲
۱-۱ شرح مسئله و اهمیت آن.....	۳
۲-۱ اهداف پروژه.....	۵
۱-۳ امکانات کلی سیستم.....	۶
۴-۱ چالش‌های پیش رو.....	۸
۵-۱ ساختار گزارش.....	۱۰
فصل دوم: تحلیل و طراحی مسئله.....	۱۱
۱-۲ مقدمه.....	۱۲
۲-۲ معماری کلی سیستم.....	۱۳
۳-۲ کاربران سیستم.....	۱۶
۴-۲ نیازمندی‌ها.....	۱۷
۵-۲ نمودار موارد کاربرد.....	۱۹
۶-۲ نمودار فعالیت.....	۲۰
۷-۲ نمودار کلاس / موجودیت‌ها.....	۲۴
۸-۲ طراحی پایگاه داده.....	۲۶
۹-۲ جمع‌بندی فصل.....	۲۸
فصل سوم: شرح پیاده‌سازی.....	۲۹
۱-۳ مقدمه.....	۳۰

۳۱	۲-۳ بستر اجرایی سیستم.....
۳۲	۳-۳ معرفی ابزارها و تکنولوژی‌ها.....
۳۶	۴-۳ پیاده‌سازی بک‌اند.....
۳۹	۵-۳ پیاده‌سازی سرویس هوش مصنوعی و چت‌بات.....
۴۲	۶-۳ پیاده‌سازی خط پردازش پایتونی اکوکاردیوگرافی.....
۶۵	۷-۳ پیاده‌سازی فرانت‌اند.....
۷۰	۸-۳ پیاده‌سازی ماژول سخت‌افزاری پایش علائم حیاتی.....
۷۲	۹-۳ جمع‌بندی فصل.....
۷۳	فصل چهارم: جمع‌بندی و نتیجه‌گیری.....
۷۴	۱-۴ جمع‌بندی نهایی.....
۷۵	۲-۴ پیشنهادهای ارائه‌شده جهت توسعه برنامه در آینده.....
۷۶	پیوست‌ها.....
۷۶	پیوست الف: متغیرهای محیطی کلیدی.....
۷۶	پیوست ب: نمودارهای تکمیلی.....
۷۹	پیوست ج: کدهای تکمیلی.....
۸۶	فهرست منابع.....
۸۸	Abstract.....

فهرست اشکال و جداول

فهرست اشکال:

- شکل ۱: معماری کلی سیستم..... ۱۴
- شکل ۲: درخت کامل ساختار سیستم..... ۱۵
- شکل ۳: نمودار موارد کاربرد سیستم..... ۱۹
- شکل ۴: نمودار فعالیت - گفتگو با دستیار هوشمند..... ۲۱
- شکل ۵: نمودار فعالیت - خط پردازش اکوکاردیوگرافی..... ۲۲
- شکل ۶: نمودار فعالیت - پایش زنده علائم حیاتی..... ۲۳
- شکل ۷: نمودار کلاس سیستم..... ۲۵
- شکل ۸: تصویر مستندات (API Swagger)..... ۳۸
- شکل ۹: تصویر رابط گفتگو با دستیار هوشمند..... ۴۱
- شکل ۱۰: تصویر اکوی بیمار در ناحیه PLAX..... ۴۳
- شکل ۱۱: تصویر اکوی بیمار در ناحیه A4C..... ۴۳
- شکل ۱۲: تصویر خام اکوی بیمار همراه با نوار قلب سبز رنگ..... ۴۴
- شکل ۱۳: تصویر نمودار ECG استخراج شده..... ۴۵
- شکل ۱۴: تصویر برآورد مقیاس از خط کش مدرج..... ۴۶
- شکل ۱۵: اندازه گیری دایامتر la در لحظه End Systol..... ۴۸
- شکل ۱۶: اندازه گیری دایامتر iva در لحظه End Diastol..... ۴۸
- شکل ۱۷: تشخیص و اندازه گیری دهلیز با بینایی کامپیوتر کلاسیک..... ۵۰
- شکل ۱۸: تشخیص و اندازه گیری بطن چپ با مدل آموزش داده شده..... ۵۱
- شکل ۱۹: نمودارهای ورودی سیستم فازی..... ۵۲
- شکل ۲۰: نمودار خروجی تجمیع شده سیستم فازی..... ۵۲
- شکل ۲۱: نمودار توالی - خط پردازش اکوکاردیوگرافی..... ۵۳
- شکل ۲۲: اطلاعات برآورد ریسک قلبی - عروقی کاربر..... ۵۳
- شکل ۲۳: خروجی تحلیل اکتشافی دیتاست Heart Disease..... ۵۴

- شکل ۲۴: خروجی تحلیل اکتشافی دیتاست Cardiovascular ۵۵
- شکل ۲۵: همبستگی ویژگی‌های نهایی با برچسب هدف در دیتاست Heart Disease ۵۷
- شکل ۲۶: همبستگی ویژگی‌های نهایی با برچسب هدف در دیتاست Cardiovascular ۵۷
- شکل ۲۷: منحنی‌های ROC مدل‌های آموزش دیده روی دیتاست Heart Disease ۵۹
- شکل ۲۸: منحنی‌های ROC مدل‌های آموزش دیده روی دیتاست Cardiovascular ۵۹
- شکل ۲۹: ماتریس درهم‌ریختگی روش‌های انسمبل روی دیتاست Heart Disease ۶۲
- شکل ۳۰: ماتریس درهم‌ریختگی روش‌های انسمبل روی دیتاست Cardiovascular ۶۲
- شکل ۳۱: تصویر داشبورد پنل مدیریت ۶۵
- شکل ۳۲: تصویر تقویم نوبت‌دهی و برنامه کاری پرسنل ۶۶
- شکل ۳۳: تصویر نوبت‌دهی ۶۶
- شکل ۳۴: تصویر پنل پزشک - تاریخچه و گزارش اکوکاردیوگرافی ۶۷
- شکل ۳۵: تصویر نمودار زنده پایش ضربان قلب ۶۸
- شکل ۳۶: تم روز فرانت ۶۹
- شکل ۳۷: تم شب فرانت ۶۹
- شکل ۳۸: سنسور MAX۳۰۱۰۲ (محل قرار گیری انگشت فرد) ۷۰
- شکل ۳۹: نمودار توالی - فرآیند نوبت‌دهی ۷۷
- شکل ۴۰: نمودار توالی - پایش علائم حیاتی ۷۷
- شکل ۴۱: نمودار حالت - جلسه پایش علائم حیاتی ۷۸
- شکل ۴۲: نمودار استقرار سیستم ۷۸

فهرست جداول:

- جدول ۱: مؤلفه‌های اصلی معماری سیستم و فناوری هر یک ۱۳
- جدول ۲: سرویس‌ها، پورت‌ها و نوع اجرای هر مؤلفه ۳۱
- جدول ۳: مقایسه‌ی عملکرد مدل‌های پایه ۶۰
- جدول ۴: مقایسه‌ی روش‌های انسمبل ۶۱

چکیده

بیماری‌های قلبی-عروقی یکی از اصلی‌ترین علل مرگ‌ومیر در جهان هستند و تشخیص زودهنگام آن‌ها از طریق اکوکاردیوگرافی، نقش مهمی در کاهش عوارض بلندمدت ایفا می‌کند. با این حال، تفسیر دستی تصاویر اکوکاردیوگرافی زمان‌بر و مستعد خطای انسانی است و کلینیک‌های تخصصی قلب نیز برای ارائه خدمات باکیفیت، نیازمند زیرساخت نرم‌افزاری یکپارچه‌ای برای مدیریت نوبت‌دهی، پرسنل و ارتباط با بیماران هستند. در این پروژه، سامانه‌ای با نام **EchoMind** طراحی و پیاده‌سازی شده است که به‌صورت هم‌زمان به این دو نیاز پاسخ می‌دهد. این سامانه یک پلتفرم کلینیکی **end-to-end** است که در قالب یک معماری میکروسرویسی، پنج مؤلفه مستقل را گرد هم می‌آورد: یک بک‌اند اصلی مبتنی بر **Laravel** برای مدیریت اداری و احراز هویت چندنقشی، یک سرویس هوش مصنوعی مبتنی بر **FastAPI** و پایگاه‌داده برداری **Qdrant** برای دستیار گفتگو محور و بازیابی اطلاعات مبتنی بر معنا، یک خط پردازش پایتونی برای تحلیل خودکار ویدیوی اکوکاردیوگرافی (طبقه‌بندی نما، استخراج رویداد چرخه قلبی، اندازه‌گیری حفرات و برآورد حجم) با بهره‌گیری از یادگیری عمیق، منطق فازی و یادگیری ماشین برای برآورد ریسک قلبی-عروقی، یک فرانت‌اند مبتنی بر **Angular** برای پنل‌های کاربری چندنقشی، و یک ماژول سخت‌افزاری مبتنی بر **ESP32** برای پایش زنده ضربان قلب از طریق پروتکل **MQTT**. نتایج این پروژه نشان می‌دهد که ترکیب هدفمند فناوری‌های متفاوت متناسب با ماهیت هر زیرمسئله، امکان ساخت سامانه‌ای یکپارچه، مقیاس‌پذیر و کارآمد را فراهم می‌کند که هم نیازهای اداری کلینیک و هم نیازهای پیشرفته تحلیل هوشمند پزشکی را پوشش می‌دهد.

کلمات کلیدی: اکوکاردیوگرافی، یادگیری عمیق، منطق فازی، اینترنت اشیا، مدیریت کلینیک،

دستیار هوشمند، یادگیری ماشین.

فصل اول: مقدمه و معرفی پروژه

۱-۱ شرح مسئله و اهمیت آن

بیماری‌های قلبی-عروقی طبق آمار سازمان‌های بهداشتی بین‌المللی، از اصلی‌ترین عوامل مرگ‌ومیر در سراسر جهان هستند و بار قابل‌توجهی بر نظام‌های سلامت و اقتصاد کشورها تحمیل می‌کنند. تشخیص زودهنگام این بیماری‌ها، از طریق ارزیابی دقیق ساختار و عملکرد قلب، نقشی تعیین‌کننده در پیشگیری از عوارض جبران‌ناپذیر و کاهش هزینه‌های درمانی طولانی‌مدت دارد. در میان روش‌های تشخیصی موجود، اکوکاردیوگرافی^۱ (سونوگرافی قلب) به دلیل غیرتهاجمی بودن، در دسترس بودن نسبی و توانایی ارزیابی هم‌زمان ساختار و عملکرد قلب، به یکی از پرکاربردترین ابزارهای تشخیصی در حوزه قلب و عروق تبدیل شده است.

با وجود این مزایا، فرآیند تفسیر تصاویر اکوکاردیوگرافی همچنان به‌شدت وابسته به مهارت، تجربه و دقت فرد متخصص است. اندازه‌گیری‌های بالینی کلیدی مانند ابعاد بطن چپ در فازهای پایان دیاستول و پایان سیستول، مساحت و حجم دهلیز چپ، و قطر مسیر خروجی بطن چپ، همگی نیازمند شناسایی دقیق فریم‌های کلیدی چرخه قلبی و اندازه‌گیری دستی مرزهای آناتومیک روی تصویر هستند. این فرآیند دستی، علاوه بر زمان‌بر بودن، در معرض تغییرپذیری بین‌فردی قرار دارد؛ به این معنا که دو متخصص مختلف ممکن است برای یک ویدیوی یکسان، اندازه‌گیری‌های اندکی متفاوت گزارش دهند. این موضوع به‌ویژه در کلینیک‌هایی که حجم بالایی از بیماران را پوشش می‌دهند، می‌تواند به کاهش کارایی و افزایش زمان انتظار بیماران منجر شود. خودکارسازی این فرآیند تحلیلی می‌تواند با کاستن از بار کاری تکراری اندازه‌گیری‌ها، زمان ارزشمند پزشک متخصص را به میزان قابل‌توجهی آزاد کرده و بر حجم بیماران قابل ویزیت در هر شیفت درمانی بیفزاید.

از منظر دیگر، صرف‌نظر از جنبه‌های تشخیصی، کلینیک‌های تخصصی قلب برای ارائه خدمات باکیفیت و مدیریت مؤثر جریان بیماران، نیازمند زیرساخت نرم‌افزاری یکپارچه‌ای هستند که فرآیندهای نوبت‌دهی، مدیریت پرسنل پزشکی و پرستاری، تعریف و مدیریت خدمات درمانی، و تخصیص منابع را

۱. Echocardiography

پوشش دهد. بسیاری از کلینیک‌های کوچک و متوسط هنوز به روش‌های سنتی مانند نوبت‌دهی تلفنی یا حضوری متکی هستند که علاوه بر کارایی پایین، تجربه کاربری ضعیفی برای بیماران رقم می‌زند و بار اداری قابل‌توجهی بر پرسنل پذیرش تحمیل می‌کند.

پروژه EchoMind دقیقاً در تقاطع این دو نیاز شکل گرفته است: از یک سو، با به‌کارگیری ترکیبی از تکنیک‌های بینایی ماشین، یادگیری عمیق، یادگیری ماشین کلاسیک و منطق فازی، فرآیند تحلیل ویدیوی اکوکاردیوگرافی را به‌صورت خودکار انجام می‌دهد و بار کاری دستی متخصص را کاهش می‌دهد؛ و از سوی دیگر، با ارائه یک پنل مدیریتی و کاربری کامل، فرآیندهای اداری و ارتباطی کلینیک را دیجیتالی، هوشمند و در دسترس‌تر می‌سازد. افزودن یک دستیار هوشمند مبتنی بر مدل‌های زبانی بزرگ و یک ماژول سخت‌افزاری پایش زنده ضربان قلب، این پلتفرم را از یک سامانه صرفاً اداری به یک اکوسیستم دیجیتال کامل برای مراقبت قلبی ارتقا می‌دهد.

۱-۲ اهداف پروژه

اهداف اصلی این پروژه که در طول فرآیند تحلیل، طراحی و پیاده‌سازی دنبال شده‌اند، به شرح زیر قابل تفکیک هستند:

۱. طراحی و پیاده‌سازی یک سامانه یکپارچه مدیریت کلینیک قلب که نوبت‌دهی آنلاین مبتنی بر تقویم تعاملی، مدیریت پرسنل و برنامه کاری آن‌ها، تعریف و مدیریت خدمات درمانی، و تخصیص منابع کلینیکی را در یک رابط کاربری واحد پوشش دهد.

۲. پیاده‌سازی سامانه احراز هویت چندنقشی و امن که با تعریف نقش‌های متمایز (بیمار، پزشک، منشی) و استفاده از پروتکل استاندارد OAuth^۲، سطوح دسترسی متفاوت و مجزایی را برای هر نقش فراهم آورد.

۳. طراحی و پیاده‌سازی خط پردازش هوشمند تحلیل ویدیوی اکوکاردیوگرافی که به‌صورت خودکار نمای تصویربرداری را طبقه‌بندی کند، رویدادهای کلیدی چرخه قلبی را استخراج نماید و اندازه‌گیری‌های بالینی حفرات قلب را با دقت قابل قبول انجام دهد.

۴. برآورد کمی ریسک بیماری‌های قلبی-عروقی با استفاده از مدل‌های یادگیری ماشین آموزش‌دیده بر پایه ویژگی‌های بالینی استاندارد بیمار، به‌عنوان مکملی برای اندازه‌گیری‌های تصویری.

۵. پیاده‌سازی دستیار هوشمند مکالمه‌محور مبتنی بر مدل زبانی بزرگ^۲ که بتواند از طریق گفتگوی طبیعی، فرآیند رزرو نوبت را هدایت کند و هم‌زمان به سوالات متداول بیماران پاسخ دقیق و به‌روز دهد.

۶. طراحی و ادغام یک ماژول سخت‌افزاری اینترنت اشیا برای پایش زنده و بلادرنگ ضربان قلب بیمار، با ارتباط پایدار و کم‌تأخیر به سامانه مرکزی از طریق پروتکل سبک MQTT.

۷. طراحی یک پنل کاربری چندنقشی با تجربه کاربری مطلوب که علاوه بر پوشش نیازهای عملکردی هر نقش، از قابلیت‌های جانبی مانند پشتیبانی چندزبانی، تم قابل تغییر، و نمایش‌های تعاملی برخوردار باشد.

۲. Large Language Model (LLM)

۳-۱ امکانات کلی سیستم

به طور خلاصه، مجموعه امکانات پیاده سازی شده در نسخه فعلی این پروژه، که در فصل سوم با جزئیات فنی بیشتری شرح داده خواهند شد، عبارت‌اند از:

- ثبت نام و ورود کاربران با دو روش مستقل: رمز عبور سستی و کد تأیید یک بار مصرف، هر دو از طریق زیرساخت احراز هویت مبتنی بر OAuth₂.
- نوبت‌دهی آنلاین با تقویم تعاملی مبتنی بر کتابخانه FullCalendar، شامل انتخاب پزشک، نوع خدمت، بازه زمانی آزاد و منبع درمانی مورد نیاز.
- مدیریت کامل پرسنل کلینیک، برنامه کاری هفتگی آن‌ها، خدمات درمانی قابل ارائه و منابع فیزیکی کلینیک.
- پنل اختصاصی پزشک برای مشاهده تقویم نوبت‌های شخصی و بررسی تاریخچه کامل معاینات اکوکاردیوگرافی هر بیمار.
- بارگذاری و پردازش کاملاً خودکار ویدیوی اکوکاردیوگرافی، شامل طبقه‌بندی نما، استخراج رویداد چرخه قلبی، اندازه‌گیری حفرات، محاسبه حجم و تولید گزارش نهایی.
- برآورد کمی ریسک بیماری قلبی-عروقی و بیماری قلبی به صورت مجزا، با استفاده از دو مدل یادگیری ماشین مستقل.
- دستیار هوشمند گفتگو محور با قابلیت تشخیص خودکار نیت پیام کاربر و پاسخ‌دهی متناسب با هر کدام.
- پایش زنده و بلادرنگ ضربان قلب با مازول سخت‌افزاری ESP₃₂ و نمایش گرافیکی آن به صورت نمودار زنده در پنل بیمار.
- پشتیبانی کامل از چندزبانگی در تمامی بخش‌های رابط کاربری، همراه با قابلیت تغییر تم بصری برنامه.

- مستندسازی خودکار API با استاندارد OpenAPI/Swagger برای تسهیل توسعه و آزمون یکپارچگی میان بخش‌های مختلف سیستم.

۴-۱ چالش‌های پیش رو

فرآیند تحلیل، طراحی و پیاده‌سازی این پروژه با چالش‌های فنی متعدد و قابل توجهی همراه بود که در ادامه هرکدام به تفصیل شرح داده می‌شوند:

پردازش تصویر پزشکی و تنوع بصری ویدیوها. ویدیوهای اکوکاردیوگرافی به دلیل ماهیت فیزیکی امواج فراصوت، دارای نویز ذاتی، کنتراست پایین در مرزهای بافتی، و تنوع بالا در زاویه و نمای تصویربرداری (مانند نماهای استاندارد A4C یعنی دیدگاه چهارحفره‌ای از راس قلب^۳ و PLAX یعنی دیدگاه طولی از کنار جناغ^۴) هستند. این تنوع، پیاده‌سازی الگوریتم‌های تشخیص خودکار رویدادهای چرخه قلبی و همچنین تعیین دقیق مقیاس پیکسل به سانتی‌متر از روی خطکش مدرج موجود در تصویر دستگاه سونوگرافی را به یکی از دشوارترین بخش‌های فنی پروژه تبدیل کرده است. برای غلبه بر این چالش، از ترکیب چند مدل تخصصی به جای یک مدل واحد و همه‌کاره استفاده شده است.

یکپارچه‌سازی چند پایگاه داده با ماهیت متفاوت. طراحی سیستم نیازمند استفاده هم‌زمان از دو نوع پایگاه داده با فلسفه متفاوت بود: پایگاه داده رابطه‌ای MySQL برای داده‌های ساخت یافته و تراکنشی که نیازمند یکپارچگی ارجاعی و روابط چندبه چند هستند؛ و پایگاه داده سندگرا MongoDB برای داده‌های پزشکی با ساختار متغیر و حجم بالای زمان-سری. مدیریت هم‌زمان این دو معماری داده در یک فریم‌ورک واحد، نیازمند استفاده از یک لایه انتزاعی بود که تجربه توسعه یکسانی برای هر دو نوع پایگاه داده فراهم کند.

ارتباط بلادرنگ و پایدار با سخت افزار در شرایط نامطمئن شبکه. طراحی پروتکل ارتباطی میان دستگاه ESP۳۲ و سرور مرکزی، باید سناریوهای واقعی و نامطلوب را نیز پوشش می‌داد: قطعی موقت اتصال Wi-Fi، خروج انگشت بیمار از روی سنسور در میانه اندازه‌گیری، و لزوم محدودسازی مدت زمان هر جلسه پایش برای جلوگیری از مصرف بی‌رویه منابع دستگاه و شبکه. این موضوع منجر به طراحی یک

۳. Apical 4-Chamber

۴. Parasternal Long Axis

ماشین حالت در سطح فریمور شد که وضعیت‌های مختلف اتصال، اندازه‌گیری و خطا را به‌صورت صریح مدیریت می‌کند.

پیاده‌سازی بازیابی اطلاعات مبتنی بر معنا بدون وابستگی کامل به سرویس ابری. برای پاسخ‌گویی دقیق و به‌روز به سوالات متداول بیماران، بدون نیاز به فراخوانی پرهزینه و کند مدل زبانی بزرگ برای هر پرسش ساده، یک لایه جستجوی معنایی محلی طراحی شد که با استفاده از embedding تولیدشده توسط یک مدل محلی و پایگاه‌داده برداری Qdrant، نزدیک‌ترین پاسخ از پیش تعریف‌شده را در کسری از زمان و بدون هزینه فراخوانی API خارجی بازمی‌گرداند.

۵-۱ ساختار گزارش

این گزارش در چهار فصل اصلی سازمان‌دهی شده است. فصل دوم به تحلیل و طراحی سیستم می‌پردازد و شامل معرفی معماری کلی، تشریح نقش‌های کاربری و نیازمندی‌های عملکردی هر نقش، و ارائه نمودارهای استاندارد تحلیل و طراحی نرم‌افزار است. فصل سوم، که مفصل‌ترین بخش گزارش را تشکیل می‌دهد، به شرح دقیق فنی پیاده‌سازی هر یک از مؤلفه‌های سیستم اختصاص دارد. در نهایت، فصل چهارم به جمع‌بندی کلی دستاوردهای پروژه و ارائه فهرستی از پیشنهادهای مشخص برای توسعه و بهبود آن در تحقیقات و نسخه‌های آتی می‌پردازد.

خلاصه فصل: در این فصل، مسئله، اهمیت آن، اهداف پروژه، امکانات کلی سیستم، چالش‌های فنی پیش رو و ساختار کلی گزارش تشریح شد. در فصل بعد، تحلیل و طراحی سیستم ارائه خواهد شد.

فصل دوم: تحلیل و طراحی مسئله

هدف این فصل، ارائه تصویری دقیق و مستند از فرآیند تحلیل نیازمندی‌ها و طراحی معماری سیستم EchoMind پیش از ورود به جزئیات پیاده‌سازی است. در ادامه، ابتدا معماری کلی سیستم و نحوه تعامل مؤلفه‌های آن با یکدیگر تشریح می‌شود، سپس کاربران سیستم و نقش‌های متمایز آن‌ها معرفی می‌گردد، نیازمندی‌های عملکردی هر نقش به تفصیل فهرست می‌شود و در نهایت نمودارهای استاندارد مهندسی نرم‌افزار شامل نمودار موارد کاربرد^۵، نمودار فعالیت^۶، نمودار کلاس^۷ و طراحی پایگاه داده^۸ ارائه خواهد شد.

۵. Use - Case Diagram

۶. Activity Diagram

۷. Class Diagram

۸. Database Diagram

۲-۲ معماری کلی سیستم

سیستم EchoMind از یک معماری میکروسرویس سبک با پنج مؤلفه اصلی مستقل تشکیل شده است. این رویکرد معماری به جای یک اپلیکیشن یکپارچه انتخاب شده تا هر مؤلفه بتواند مستقل از سایرین توسعه، آزمون، به روزرسانی و در صورت نیاز مقیاس داده شود؛ برای نمونه، بار پردازشی سنگین خط پردازش اکو هیچ تأثیری بر کارایی و پاسخ‌گویی بک‌اند اصلی که مسئول عملیات اداری روزمره است، نمی‌گذارد.

جدول ۱: مؤلفه‌های اصلی معماری سیستم و فناوری هر یک

مؤلفه	فناوری اصلی	مسیر در مخزن	مسئولیت کلیدی
بک‌اند اصلی	Laravel ۱۲ / PHP ۸.۴	back/laravel	احراز هویت، نوبت‌دهی، مدیریت پرسنل و خدمات، هماهنگی چت‌بات
بروکر پیام	Mosquitto	mqtt-broker	انتقال بلادرنگ داده بین دستگاه ESP۳۲ و بک‌اند
سرویس هوش مصنوعی	FastAPI + Qdrant	back/faiss	مسیریابی نیت گفتگو و جستجوی معنایی
خط پردازش اکو	Python	back/python_echo	تحلیل ویدیوی اکوکارديوگرافی و برآورد ریسک قلبی
فرانت‌اند	Angular ۲۰	front	پنل‌های کاربری چندنقشی و رابط گرافیکی
دروازه API	Kong Gateway	kong-gateway	ثبت لاگ و مانیتورینگ ترافیک عبوری از API (نقش محدود و مکمل)

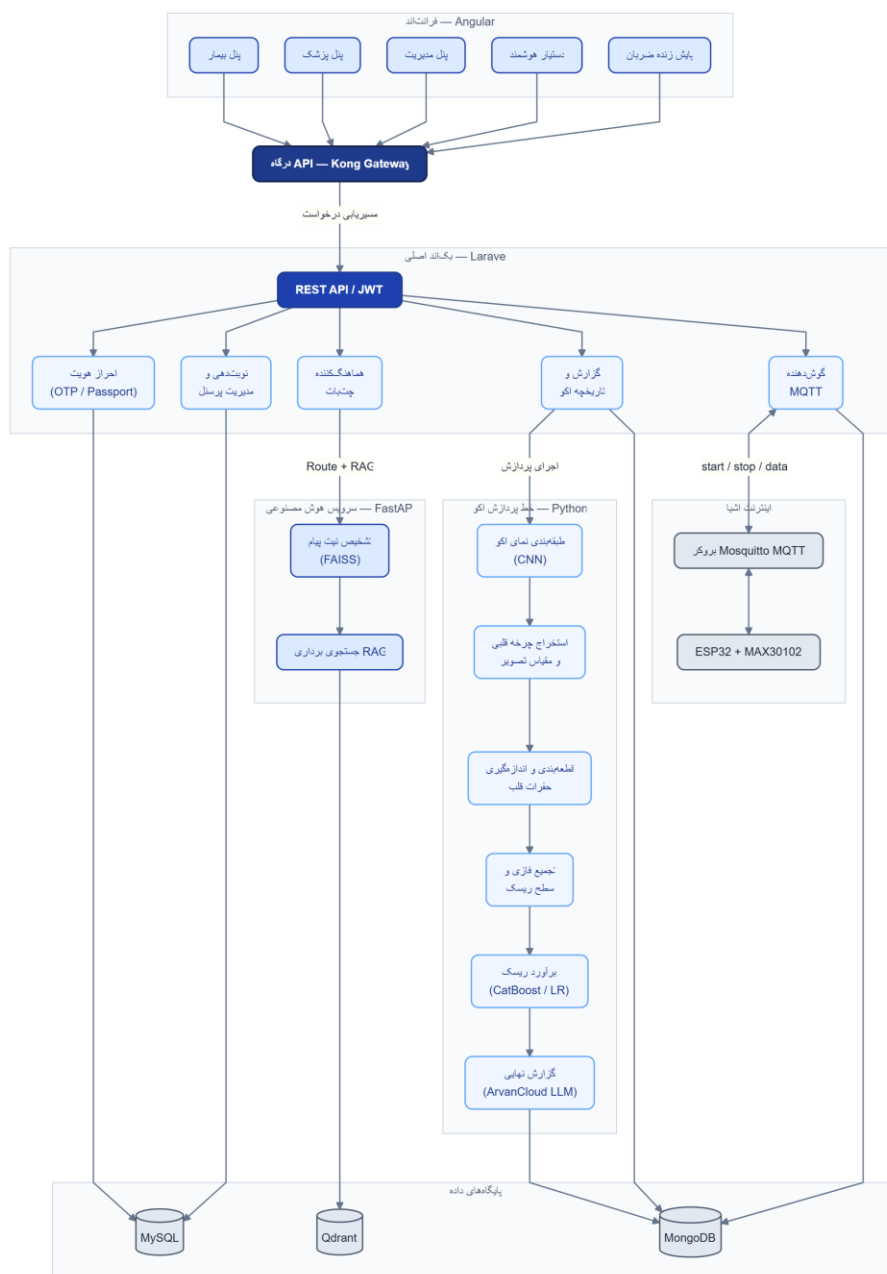
نحوه تعامل این مؤلفه‌ها با یکدیگر به شرح زیر است: فرانت‌اند Angular به عنوان تنها نقطه ورود کاربر، از طریق فراخوانی‌های REST API با بک‌اند Laravel در ارتباط است. بک‌اند Laravel به عنوان هماهنگ‌کننده مرکزی، در صورت نیاز به پاسخ هوشمند برای دستیار گفتگو محور، درخواست را به سرویس مستقل FAISS ارجاع می‌دهد تا نیت پیام تشخیص داده شود و در صورت لزوم جستجوی معنایی در پایگاه دانش انجام گیرد؛ برای تولید متن پاسخ نهایی نیز از سرویس‌های مدل زبانی بزرگ ابری استفاده می‌شود.

در مسیر سخت‌افزاری، دستگاه ESP۳۲ داده ضربان قلب اندازه‌گیری‌شده را از طریق پروتکل سبک MQTT به بروکر Mosquitto منتشر می‌کند و یک فرمان زمینه‌ای در سمت Laravel به‌صورت دائمی روی این موضوعات مشترک است و داده دریافتی را در پایگاه‌داده MongoDB ذخیره می‌کند. در نهایت، خط پردازش پایتونی اکو، برخلاف سایر مؤلفه‌ها، به‌صورت یک سرویس همیشه‌فعال طراحی نشده است؛ بلکه به‌عنوان یک ابزار خط‌فرمان مستقل، به‌ازای هر بیمار و هر ویدیوی جدید به‌صورت مجزا فراخوانی می‌شود.



شکل ۱: معماری کلی سیستم

این تصمیم طراحی — یعنی اجرای خط پردازش اکو به‌صورت ابزار خط‌فرمان به‌جای سرویس دائمی — با توجه به ماهیت غیربلادرنگ این فرآیند و همچنین سادگی به‌کارگیری منابع محاسباتی سنگین تنها در زمان نیاز واقعی، اتخاذ شده است.



شکل ۲: درخت کامل ساختار سیستم

علاوه بر پنج مؤلفه اصلی فوق، دروازه Kong API نیز در معماری پروژه استفاده شده است؛ با این حال، نقش این مؤلفه در وضعیت فعلی سیستم محدود و صرفاً در حد ثبت لاگ و مشاهده ترافیک عبوری از API است و هنوز به‌عنوان یک لایه فعال مسیریابی یا کنترل دسترسی در مسیر اصلی درخواست‌ها قرار نگرفته است. بنابراین می‌توان آن را در وضعیت فعلی، مؤلفه‌ای جانبی و آماده برای توسعه‌های آتی (مانند نرخ‌گذاری درخواست یا تجمع لاگ چند سرویس) در نظر گرفت.

۳-۲ کاربران سیستم

سیستم دارای سه نقش کاربری کاملاً متمایز است که در شمارنده UserRole در سمت بک‌اند

تعریف شده‌اند و هرکدام سطح دسترسی و مجموعه عملیات مجاز خاص خود را دارند:

بیمار: پایه‌ای‌ترین و پرتعدادترین نقش کاربری سیستم است. بیمار می‌تواند نوبت رزرو کند،

تاریخچه و گزارش‌های اکوکاردیوگرافی خود را مشاهده کند، با دستیار هوشمند گفتگو کند، و جلسه پایش

زنده ضربان قلب خود را آغاز و مدیریت نماید.

پزشک: دسترسی به پنل اختصاصی پزشک دارد که شامل مشاهده نوبت‌های تعیین‌شده خود در

قالب تقویم و بررسی کامل تاریخچه معاینات اکوکاردیوگرافی هر بیمار است.

منشی: معادل نقش پذیرش در کلینیک‌های فیزیکی است و مسئولیت مدیریت کامل پرسنل،

خدمات درمانی، منابع کلینیکی و نوبت دهی و پایش علائم حیاتی سازمان را بر عهده دارد

۲-۴ نیازمندی‌ها

نیازمندی‌های عملکردی سیستم بر اساس نقش کاربری دسته‌بندی و در ادامه به تفصیل فهرست شده‌اند.

۲-۴-۱ نیازمندی‌های کاربر عادی (بیمار)

- ثبت نام و ورود به سیستم از طریق دو مسیر مستقل: رمز عبور سستی، یا کد تأیید یک بار مصرف ارسالی به ایمیل.
- مشاهده فهرست پزشکان، خدمات درمانی قابل ارائه، و بازه‌های زمانی خالی، به همراه امکان رزرو نوبت جدید.
- مشاهده و ویرایش پروفایل شخصی و اطلاعات بالینی پایه.
- گفتگو با دستیار هوشمند سیستم به منظور رزرو نوبت به صورت مکالمه‌ای طبیعی یا دریافت پاسخ سوالات متداول.
- شروع و پایان جلسه پایش زنده ضربان قلب و مشاهده نمودار زنده ضربان
- مشاهده تاریخچه کامل و گزارش نهایی تحلیل شده هر معاینه اکوکاردیوگرافی.

۲-۴-۲ نیازمندی‌های پزشک

- مشاهده تقویم نوبت‌های تعیین شده شخصی به تفکیک روز، هفته و ماه.
- دسترسی به تاریخچه کامل معاینات اکوکاردیوگرافی هر بیمار، شامل تصاویر کلیدی استخراج شده و گزارش تحلیل نهایی.
- مدیریت تنظیمات پروفایل شخصی، شامل اطلاعات تماس و تخصص.

۲-۴-۳ نیازمندی‌های مدیریت سیستم

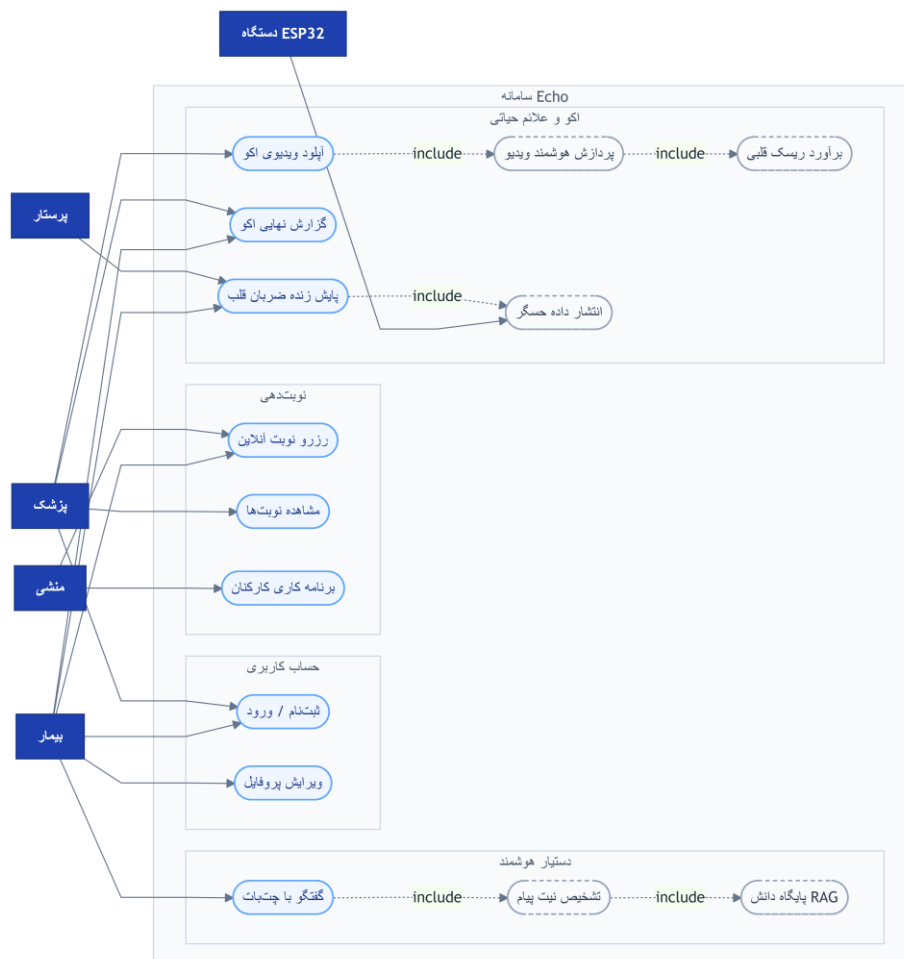
- مدیریت کامل پرسنل کلینیک (افزودن، ویرایش، غیرفعال‌سازی) و تعریف برنامه کاری هفتگی هر یک از آنان.

- مدیریت خدمات درمانی قابل ارائه و منابع فیزیکی کلینیک.

- مدیریت فهرست تخصص‌های پزشکی موجود در کلینیک.

۵-۲ نمودار موارد کاربرد

نمودار موارد کاربرد کامل سیستم به صورت گرافیکی ترسیم شده است. در ادامه، خلاصه‌ای متنی از اکتورها و مهم‌ترین موارد کاربرد ارائه می‌شود: اکتورها شامل بیمار، پزشک، منشی و دستگاه ESP32 به عنوان یک اکتور سیستمی خارجی هستند. موارد کاربرد کلیدی بیمار عبارت‌اند از ثبت نام/ورود، ویرایش پروفایل، رزرو نوبت آنلاین، گفتگو با دستیار هوشمند، شروع پایش زنده ضربان قلب و مشاهده گزارش نهایی اکو. موارد کاربرد کلیدی مدیریت شامل مدیریت پرسنل و تخصص‌ها، مدیریت برنامه کاری، مدیریت خدمات و منابع است. موارد کاربرد سیستمی که توسط سایر موارد کاربرد فراخوانی می‌شوند عبارت‌اند از طبقه‌بندی نیت پیام کاربر، جستجوی معنایی در پایگاه دانش، پردازش هوشمند ویدیوی اکو، برآورد ریسک قلبی-عروقی، و انتشار داده حسگر از طریق پروتکل MQTT.



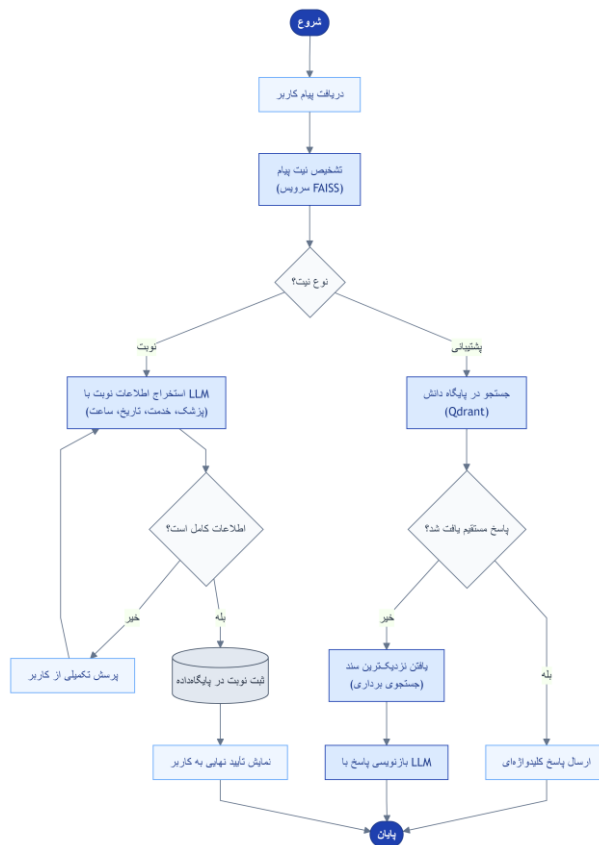
شکل ۳: نمودار موارد کاربرد سیستم

۶-۲ نمودار فعالیت

سه فرآیند کلیدی و پرتکرار سیستم در قالب نمودار فعالیت مدل‌سازی شده‌اند. در ادامه شرح گام‌به‌گام هر فرآیند ارائه می‌شود.

۶-۲-۱ فرآیند رزرو نوبت از طریق دستیار هوشمند

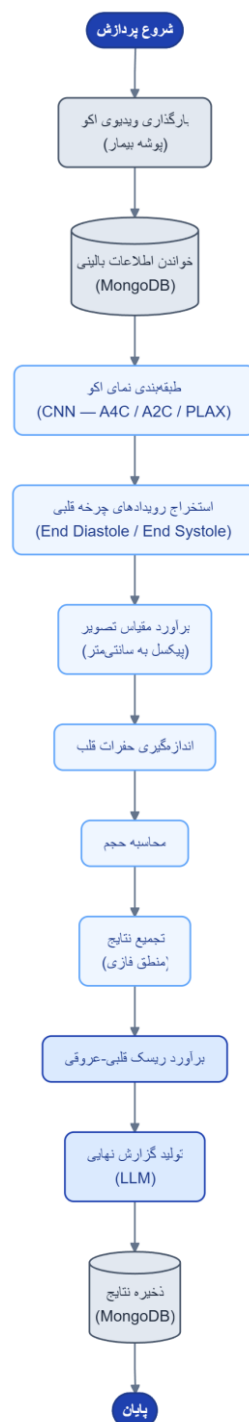
این فرآیند با ارسال پیام آزاد کاربر آغاز می‌شود. در گام نخست، سرویس FAISS نیت پیام را با ترکیبی از تطبیق کلیدواژه و جستجوی برداری تشخیص می‌دهد. در صورتی که نیت پیام «رزرو نوبت» تشخیص داده شود، مدل زبانی بزرگ وارد عمل شده و به‌صورت گام‌به‌گام، اطلاعات لازم (نام یا تخصص پزشک، نوع خدمت، تاریخ و ساعت مدنظر) را از پیام‌های متوالی کاربر استخراج می‌کند. اگر در هر مرحله اطلاعاتی ناقص باشد، سیستم سوال تکمیلی مشخصی را به کاربر باز می‌گرداند. پس از تکمیل کامل اطلاعات، نوبت از طریق سرویس داخلی نوبت‌دهی در پایگاه داده ثبت می‌شود و پیام تأیید نهایی به کاربر نمایش داده می‌شود. در مقابل، اگر نیت پیام «پشتیبانی یا سوال متداول» تشخیص داده شود، مسیر جایگزینی طی می‌شود که در آن ابتدا جستجوی برداری در پایگاه دانش Qdrant انجام و در صورت یافتن پاسخ مستقیم کلیدواژه‌ای، همان بازگردانده می‌شود؛ در غیر این صورت، نزدیک‌ترین سند مرتبط یافته شده و توسط مدل زبانی بزرگ به عبارتی روان و طبیعی بازنویسی می‌شود.



شکل ۴: نمودار فعالیت - گفتگو با دستیار هوشمند

۲-۶-۲ فرآیند پردازش ویدیوی اکوکاردیوگرافی

این فرآیند زمانی آغاز می‌شود که پزشک یا اپراتور، ویدیوی معاینه اکوکاردیوگرافی بیمار را در پوشه اختصاصی وی قرار می‌دهد و خط پردازش پایتونی روی آن اجرا می‌شود. ابتدا اطلاعات بالینی پایه بیمار از پایگاه داده MongoDB خوانده می‌شود. سپس نمای تصویربرداری ویدیو با استفاده از یک مدل شبکه عصبی کانولوشنی کلاس بندی می‌شود. در گام بعد، رویدادهای کلیدی چرخه قلبی استخراج می‌شوند و مقیاس پیکسل به سانتی متر تصویر برآورد می‌گردد. با در دست داشتن این اطلاعات، اندازه گیری دقیق حفرات قلب و مسیر خروجی بطن انجام می‌شود. در گام بعدی، تمامی این نتایج عددی توسط یک لایه منطق فازی تجمیع و به یک ارزیابی کیفی و کمی نهایی تبدیل می‌شوند. هم زمان، مدل های یادگیری ماشین بر اساس ویژگی های بالینی بیمار، ریسک قلبی-عروقی را برآورد می کنند. در گام پایانی، گزارش نهایی تولید و در مسیر خروجی مشترک با Laravel و پایگاه داده MongoDB ذخیره می گردد.

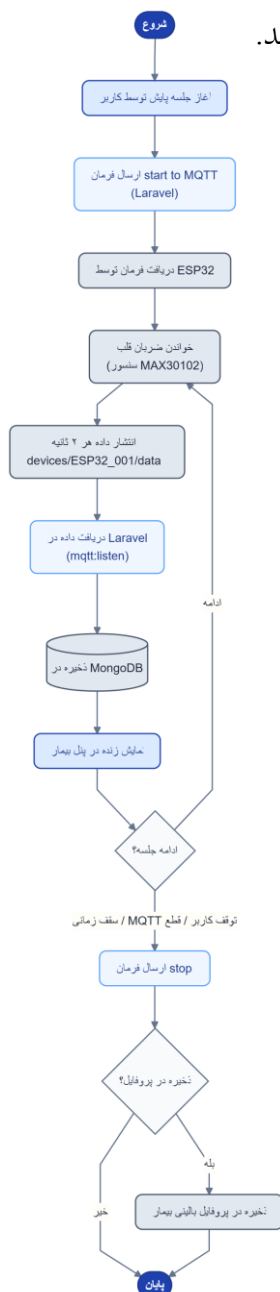


شکل ۵: نمودار فعالیت - خط پردازش اکوکاردیوگرافی

۲-۶-۳ فرآیند پایش زنده ضربان قلب

این فرآیند با آغاز جلسه پایش توسط کاربر از طریق پنل کاربری شروع می‌شود. درخواست شروع، از طریق بک‌اند **Laravel** به بروکر **MQTT** و از آنجا به دستگاه **ESP۳۲** ارسال می‌شود. دستگاه پس از دریافت فرمان، به صورت پیوسته و هر دو ثانیه یکبار، مقدار ضربان قلب اندازه‌گیری شده توسط سنسور **MAX۳۰۱۰۲** را در قالب یک پیام **JSON** منتشر می‌کند. فرمان گوش‌ده در سمت **Laravel** این داده‌ها را

دریافت کرده و در پایگاه داده MongoDB ذخیره می کند. پنل کاربری بیمار نیز به صورت موازی و با فاصله زمانی مشابه، از طریق فراخوانی API داده های جدید را واکنشی کرده و در قالب نمودار زنده نمایش می دهد. جلسه پایش تا زمانی ادامه می یابد که کاربر آن را به صورت دستی پایان دهد، مدت زمان جلسه از سقف مجاز عبور کند، یا ارتباط MQTT بیش از بیست ثانیه قطع شود. در پایان، کاربر می تواند نتیجه نهایی جلسه را در پروفایل بالینی خود ذخیره کند.



شکل ۶: نمودار فعالیت - پایش زنده علائم حیاتی

۷-۲ نمودار کلاس / موجودیت‌ها

مهم‌ترین موجودیت‌های سیستم به شرح زیر هستند:

کاربر^۹: نماینده هر کاربر سیستم است و شامل نام، ایمیل، نقش و رمز عبور رمزنگاری شده است. این موجودیت با استفاده از **Laravel Passport**، توکن‌های **OAuth۲** برای احراز هویت **API** صادر می‌کند.

نقش کاربر^{۱۰}: یک شمارنده سه‌مقداره که نقش‌های کاربر، پزشک و منشی را تعریف می‌کند.^{۱۱}
کارمند^{۱۲}، متخصص^{۱۳}، سرویس^{۱۴}، منبع^{۱۵}: موجودیت‌های پایه‌ای مربوط به مدیریت اداری کلینیک که هرکدام جدول مستقل خود را در **MySQL** دارند و از طریق کلید خارجی به موجودیت نوبت^{۱۶} مرتبط می‌شوند.

نوبت: قلب مدل داده‌ای بخش نوبت‌دهی است و شامل ارجاع به کاربر رزروکننده، پزشک، نوع خدمت و بازه زمانی است. این موجودیت همچنین رابطه چندبه‌چند با موجودیت منبع دارد.

اطلاعات بیمار^{۱۷}: پروفایل بالینی گسترده بیمار که در **MongoDB** ذخیره می‌شود و شامل ویژگی‌های بالینی استاندارد (سن، جنسیت، قد، وزن، فشار خون، کلسترول، ضربان قلب حداکثر) است.

۹ . User

۱۰ .User Role

۱۱ .admin,user,doctor,nurse,operator,monshi

۱۲ .Staff

۱۳ .Expertise

۱۴ .Service

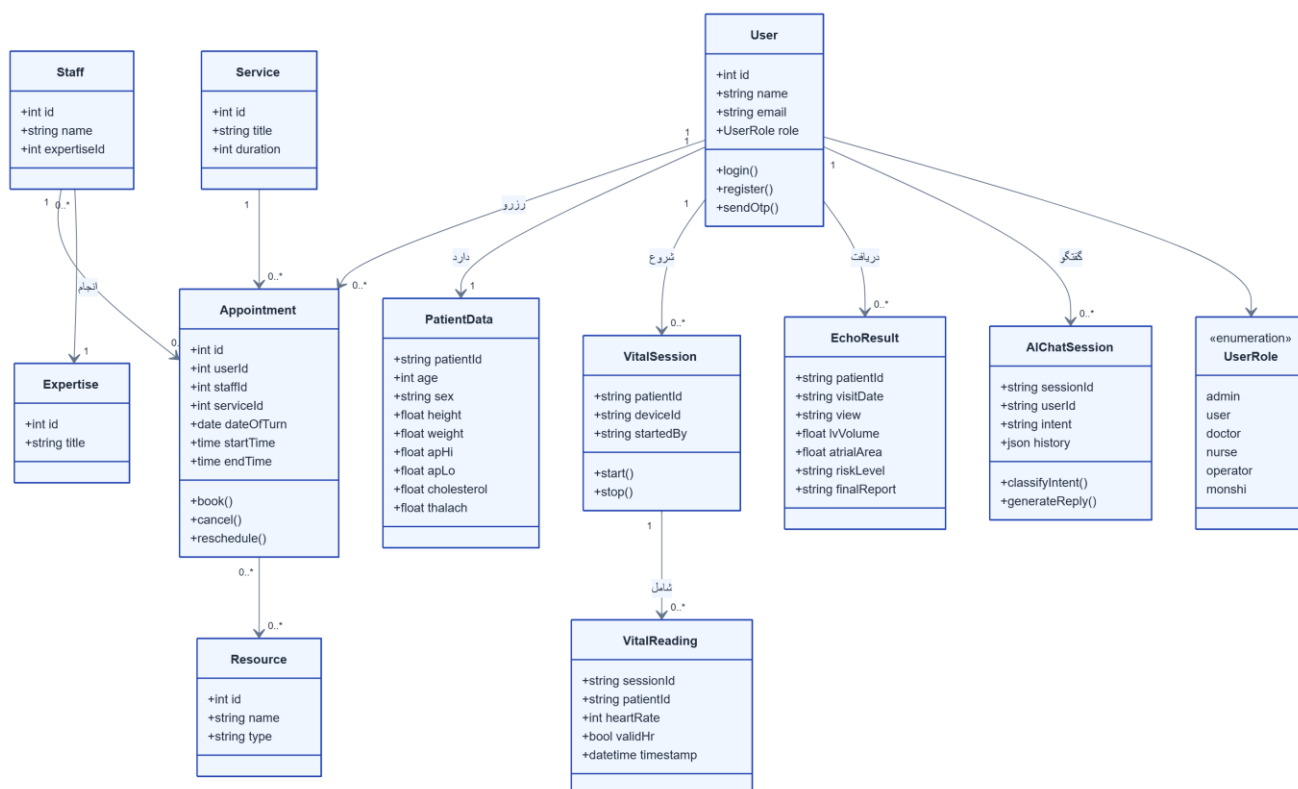
۱۵ .Resource

۱۶ .Appointment

۱۷ . PatientData

نتیجه اکو^{۱۸}: خروجی نهایی خط پردازش اکو که شامل شناسه بیمار، تاریخ ویزیت، نمای تشخیص داده شده، حجم بطن چپ، مساحت دهلیزی، سطح ریسک نهایی و متن گزارش تولید شده است.

دستیار هوشمند^{۱۹}: نماینده یک جلسه گفتگوی کاربر با دستیار هوشمند، شامل شناسه کاربر، نیت تشخیص داده شده و تاریخچه کامل مکالمه است.



شکل ۷: نمودار کلاس سیستم

۱۸. EchoResult

۱۹. AIChatSession

۲-۸ طراحی پایگاه داده

طراحی لایه داده سیستم به صورت آگاهانه و بر پایه ماهیت متفاوت انواع داده، از سه فناوری پایگاه داده مجزا بهره می برد.

MySQL (رابطه‌ای، از طریق Eloquent ORM) این پایگاه داده برای نگهداری داده‌های ساخت یافته، رابطه‌ای و تراکنشی سیستم به کار رفته است؛ از جمله جداول `users`، `staffs`، `services`، `appointments`، `expertises`، `resources`، جدول واسط `appointment_resources`، و مجموعه جداول استاندارد OAuth مربوط به Passport Laravel. انتخاب یک پایگاه داده رابطه‌ای برای این بخش، مستقیماً از نیاز به یکپارچگی ارجاعی^{۲۰} قوی، روابط چندبه چند صریح، و پشتیبانی از تراکنش‌های ACID در فرآیند حساس نوبت‌دهی ناشی می شود.

MongoDB (سندگرا، از طریق پکیج `jenssegers/mongodb`) این پایگاه داده برای نگهداری داده‌های پزشکی با ساختار متغیر و حجم بالا در نظر گرفته شده است؛ از جمله مجموعه‌های `patient_profiles` که اطلاعات بالینی گسترده و گاه اختیاری بیمار را در بر می گیرد، و `vital_readings` که داده‌های زمان-سری با نرخ تولید بالا هستند. انتخاب یک پایگاه داده سندگرا برای این بخش، از دو ملاحظه اصلی نشأت می گیرد: نخست، تنوع بالای فیلدهای اختیاری در پروفایل بالینی بیمار؛ و دوم، ماهیت انباشتی و با نرخ بالای داده‌های زمان-سری علائم حیاتی.

Qdrant (پایگاه داده برداری، در حالت محلی/فایلی) این پایگاه داده تخصصی برای ذخیره بردارهای `embedding` معنایی متن پایگاه دانش کلینیک به کار رفته و توسط سرویس مستقل FAISS جهت انجام جستجوی شباهت برداری در فرآیند بازیابی اطلاعات مبتنی بر معنا مورد استفاده قرار می گیرد. در پیکربندی فعلی، Qdrant در حالت اجرای محلی و فایلی به کار گرفته شده که برای مقیاس فعلی

۲۰. Referential Integrity

پروژه انتخاب مناسبی است، هرچند این تصمیم محدودیت‌هایی نیز به همراه دارد که در فصل چهارم به آن‌ها اشاره خواهد شد.

۹-۲ جمع‌بندی فصل

در این فصل، معماری میکروسرویسی سیستم و منطق پشت این انتخاب معماری، نقش‌های کاربری متمایز و نیازمندی‌های عملکردی هر یک، و در نهایت نمودارهای استاندارد تحلیل و طراحی نرم‌افزار به تفصیل تشریح شد. رویکرد ترکیبی به کارگیری سه فناوری پایگاه داده مجزا، انعطاف و کارایی لازم برای مدیریت هم‌زمان داده‌های اداری ساخت‌یافته، داده‌های پزشکی با ساختار متغیر، و داده‌های معنایی متنی را فراهم کرده است. در فصل بعد، جزئیات دقیق فنی پیاده‌سازی هر یک از این مؤلفه‌های معماری شرح داده خواهد شد.

فصل سوم: شرح پیاده‌سازی

این فصل به شرح دقیق و فنی پیاده‌سازی هر یک از مؤلفه‌های سیستم EchoMind اختصاص دارد. تمرکز اصلی این فصل بر تشریح بخش‌ها، الگوهای طراحی، تصمیمات فنی کلیدی و کدهای اصلی هر مؤلفه است؛ توضیح خط‌به‌خط تمامی کدهای پروژه یا شرح تمامی صفحات و فرم‌های رابط کاربری در حوصله این گزارش نیست، بلکه بخش‌ها و منطق‌های مهم و تعیین‌کننده هر لایه معرفی می‌شوند. ترتیب ارائه مطالب این فصل، از بستر اجرایی و ابزارهای زیرساختی آغاز شده و سپس به ترتیب هر یک از پنج مؤلفه اصلی معماری پوشش داده می‌شود.

۲-۳ بستر اجرایی سیستم

سیستم EchoMind به صورت مجموعه ای از سرویس های مستقل و مجزا، هرکدام با پورت اختصاصی خود، اجرا می شود که این جداسازی، امکان توسعه و اشکال زدایی مستقل هر بخش را فراهم می آورد:

جدول ۲: سرویس ها و نوع اجرای هر مؤلفه

سرویس	نوع اجرا
بک اند Laravel	سرور HTTP دائمی
سرویس FAISS (FastAPI)	سرور HTTP دائمی
بروکر MQTT (Mosquitto)	سرویس دائمی (کانتینر Docker)
فرانت اند Angular (توسعه)	سرور توسعه Angular CLI
خط پردازش اکو (Python)	اجرای یک باره خط فرمان به ازای هر ویدیو
Kong Api Gateway	سرویس دائمی (کانتینر Docker)

۳-۳ معرفی ابزارها و تکنولوژی‌ها

در این بخش، هر یک از فناوری‌ها و کتابخانه‌های کلیدی مورد استفاده در پروژه، به همراه دلیل انتخاب آن‌ها، معرفی می‌شوند.

۳-۳-۱ فریم‌ورک لاراول^{۲۱} (الگوی MVC)

بک‌اند اصلی سیستم با نسخه ۱۲ فریم‌ورک Laravel (بر پایه PHP نسخه ۸.۴) پیاده‌سازی شده است. Laravel با پیروی از الگوی معماری Model-View-Controller، سیستم مسیریابی خوانا و انعطاف‌پذیر، ابزار ORM با نام Eloquent، و اکوسیستم گسترده‌ای از بسته‌های جانبی، بستر مناسبی برای توسعه سریع و قابل‌نگهداری یک API پیچیده فراهم می‌آورد.

۳-۳-۲ پاسپورت لاراول^{۲۲}

احراز هویت لایه API سیستم با استفاده از بسته رسمی Laravel Passport که پیاده‌سازی کاملی از پروتکل استاندارد صنعتی OAuth۲ است، انجام شده است. این انتخاب امکان صدور توکن دسترسی امن و قابل‌ابطال برای هر کاربر، پشتیبانی از چرخه حیات کامل توکن، و کنترل دسترسی مبتنی بر میان‌افزار در سطح هر مسیر API را فراهم می‌کند.

۳-۳-۳ دیتابیس مونگو در لاراول^{۲۳} (mongodb)

برای امکان استفاده از Eloquent ORM روی پایگاه‌داده MongoDB، از این بسته انتزاعی استفاده شده است. این انتخاب اجازه می‌دهد مدل‌هایی مانند PatientData و VitalReading با همان نحو آشنای Eloquent تعریف و پرس‌وجو شوند، در حالی که در پس‌زمینه از انعطاف ساختار سندی MongoDB بهره می‌برند.

۲۱. Laravel Framework

۲۲. Laravel Passport

۲۳. Laravel MongoDB

۳-۳-۴ فریمورک انگولار^{۲۴}

فرانت‌اند سیستم با نسخه ۲۰ فریم‌ورک Angular و با استفاده کامل از معماری کامپوننت‌های مستقل پیاده‌سازی شده است. انتخاب Angular برای این پروژه، با توجه به معماری ماژولار و مقیاس‌پذیر آن، ابزار خط‌فرمان قدرتمند برای تولید و مدیریت کد، و پشتیبانی رسمی و یکپارچه از TypeScript صورت گرفته است.

۳-۳-۵ تقویم تعاملی^{۲۵}

پیاده‌سازی تقویم تعاملی نوبت‌دهی و نمایش برنامه کاری هفتگی کارکنان، با استفاده از پکیج رسمی FullCalendar برای Angular انجام شده است. این کتابخانه امکان نمایش رویدادها در قالب‌های مختلف، ایجاد و ویرایش تعاملی رویدادها با کشیدن و رهاکردن، و سفارشی‌سازی کامل ظاهر و رفتار تقویم را فراهم می‌کند.

۳-۳-۶ نمایش مدل سه‌بعدی قلب^{۲۶}

برای ارائه یک تجربه بصری جذاب‌تر در صفحه معرفی عمومی سایت، از کتابخانه three.js برای رندر و نمایش تعاملی یک مدل سه‌بعدی قلب در کامپوننت heart-visualization استفاده شده است.

۳-۳-۷ فریمورک FastAPI و دیتابیس Qdrant

سرویس مستقل هوش مصنوعی با فریم‌ورک FastAPI پیاده‌سازی شده که به دلیل کارایی بالای آن در پردازش هم‌زمان، و اعتبارسنجی قوی داده ورودی/خروجی، انتخاب مناسبی برای یک سرویس استنتاج مدل با ترافیک هم‌زمان است. Qdrant نیز به‌عنوان یک پایگاه‌داده برداری تخصصی و سبک، با

۲۴. Angular Framework

۲۵. FullCalendar

۲۶. Three.js

قابلیت اجرا در حالت کاملاً محلی و فایلی، برای ذخیره و جستجوی بردارهای embedding به کار رفته است.

۸-۳-۳ مدل Sentence-Transformer (Embedding محلی)

برای تبدیل متن سوالات کاربر و اسناد پایگاه دانش کلینیک به بردارهای عددی معنایی، از یک مدل محلی مبتنی بر معماری Transformer با راهبرد تجميع MeanPooling استفاده شده است. اجرای این مدل به صورت کاملاً محلی، هزینه عملیاتی سیستم را کاهش داده و تأخیر پاسخ‌گویی را به طور قابل توجهی بهبود می‌بخشد.

۹-۳-۳ دروازه هوش مصنوعی ابر آروان^{۲۷} و AIAS^{۲۸}

سرویس AI as a Service ابر آروان به عنوان زیرساخت هوش مصنوعی سامانه برای تولید پاسخ‌های زبانی طبیعی در دستیار گفت‌وگو محور و گزارش‌نویسی نهایی خط پردازش اکو استفاده شده است. انتخاب این سرویس با هدف افزایش پایداری، کاهش وابستگی به سرویس‌های خارجی و حفظ دسترسی در شرایط ناپایداری شبکه انجام گرفته است.

۱۰-۳-۳ مدل‌های یادگیری عمیق PyTorch و Keras

خط پردازش تصویر اکو از مدل‌های از پیش آموزش دیده در چهارچوب یادگیری عمیق و روش‌های کلاسیک مبانی بینایی استفاده شده.

^{۲۷}. Arvan Cloud AI Gateway

^{۲۸}. Ai As Service

۱۱-۳-۳ رگرسیون لجستیک و CatBoost

برای برآورد کمی ریسک، دو مدل یادگیری ماشین کلاسیک و مستقل به کار رفته‌اند: یک مدل CatBoost برای برآورد ریسک قلبی-عروقی و یک مدل رگرسیون لجستیک برای برآورد ریسک بیماری قلبی. هر دو مدل بر پایه مجموعه‌ای از ویژگی‌های بالینی استاندارد بیمار از پیش آموزش دیده‌اند.

۱۲-۳-۳ منطق فازی^{۲۹}

برای تجمیع نتایج عددی حاصل از چندین اندازه‌گیری مستقل حفرات قلب و تبدیل آن‌ها به یک ارزیابی نهایی واحد و قابل فهم، از یک لایه منطق فازی استفاده شده است. انتخاب منطق فازی به جای یک آستانه‌گذاری ساده یا میانگین‌گیری خطی، از این واقعیت نشأت می‌گیرد که اندازه‌گیری‌های تصویری پزشکی ذاتاً دارای عدم قطعیت هستند.

۱۳-۳-۳ اجزای سخت افزار پایش علائم حیاتی^{۳۰}

ارتباط داده میان دستگاه ESP^{۳۲} و سرور مرکزی، از طریق پروتکل پیام‌رسانی سبک MQTT^{۳۱} برقرار می‌شود. این پروتکل به دلیل سربار پروتکلی بسیار کم، مدل انتشار/اشتراک مناسب برای ارتباطات و سازگاری بالا با دستگاه‌های دارای منابع محاسباتی محدود، برای این بخش انتخاب شده است.

^{۲۹}. Fuzzy Logic

^{۳۰}. MQTT, Mosquitto, PubSubClient

^{۳۱}. Message Queuing Telemetry Transport

۳-۴ پیاده سازی بک اند

بک اند اصلی سیستم به عنوان هماهنگ کننده مرکزی، مسئولیت های متعددی از جمله احراز هویت، مدیریت نوبت دهی، مدیریت پرسنل و خدمات، هماهنگی جریان چت بات، و مدیریت داده های پزشکی و علائم حیاتی را بر عهده دارد. در ادامه هر یک از این زیرسیستم ها به تفصیل شرح داده می شود.

۳-۴-۱ احراز هویت

کنترلر **AuthController** هسته اصلی زیرسیستم احراز هویت است و دو روش مستقل ورود را پشتیبانی می کند: ورود سنتی با ترکیب ایمیل و رمز عبور، و ورود مبتنی بر کد تأیید یک بار مصرف که در آن ابتدا کد تصادفی به کاربر ارسال شده و سپس از طریق مسیر مجزای دیگری اعتبارسنجی می شود. صدور و اعتبارسنجی توکن دسترسی از طریق **Laravel Passport** و بر پایه استاندارد **OAuth2** انجام می شود؛ پس از ورود موفق، نقش کاربر در پاسخ ورود بازگردانده می شود تا فرانت اند بتواند بلافاصله مسیرها و دسترسی های متناسب با آن نقش را در رابط کاربری فعال کند.

۳-۴-۲ مدیریت نوبت دهی

کنترلر **AppointmentController** قلب زیرسیستم نوبت دهی است و مسئولیت های زیر را پوشش می دهد: اعلام بازه های زمانی آزاد بر اساس برنامه کاری پزشک منتخب و نوبت های از پیش رزرو شده، رزرو نوبت جدید با در نظر گرفتن هم زمان پزشک، نوع خدمت، منابع مورد نیاز و همچنین عملیات ویرایش و لغو نوبت های موجود. این کنترلر برای جلوگیری از تداخل زمانی، پیش از ثبت نهایی هر نوبت، اعتبارسنجی تداخل با نوبت های موجود همان پزشک و بازه زمانی را در سطح پایگاه داده انجام می دهد.

۳-۴-۳ نمونه کد: تعریف مسیر API نوبت دهی

به عنوان نمونه ای از نحوه تعریف مسیرهای API در **Laravel**، بخشی از فایل **routes/api.php**

مربوط به زیرسیستم نوبت دهی در زیر آورده شده است:

```
Route::group(['prefix' => 'appointment'], function () {
    Route::post('get-available-slots', [AppointmentController::class,
'getAvailableSlots'])->middleware('auth:api');
    Route::post('add-appointment', [AppointmentController::class,
'addAppointment'])->middleware('auth:api');
    Route::patch('edit-appointment/{id}',
[AppointmentController::class, 'editAppointment'])->
>middleware('auth:api');
    Route::get('get-appointment/{id}', [AppointmentController::class,
'getAppointment'])->middleware('auth:api');
    Route::post('get-calendar-dashboard',
[AppointmentController::class, 'getCalendarDashboard'])->
>middleware('auth:api');
    Route::get('get-current-appointment',
[AppointmentController::class, 'getCurrentAppointment'])->
>middleware('auth:api');
    Route::post('list', [AppointmentController::class,
'listAppointments'])->middleware('auth:api');
});
```

۳-۴-۴ مدیریت داده‌های پزشکی و گزارش‌ها

کنترلر **PatientDataController** مسئول ثبت اولیه و به‌روزرسانی پروفایل بالینی بیمار است و اعتبارسنجی محدوده مقادیر بالینی را در همین لایه انجام می‌دهد. کنترلرهای **EchoHistoryController** و **PatientReportController** مسئول واکنشی نتایج و گزارش‌های تولیدشده توسط خط پردازش پایتونی، از مسیر خروجی فایل سیستمی مشترک، و ارائه آن‌ها در قالب پاسخ **API** قابل‌مصرف برای فرانت‌اند هستند. این جداسازی معماری امکان می‌دهد بار پردازشی سنگین کاملاً از بک‌اند اصلی جدا بماند.

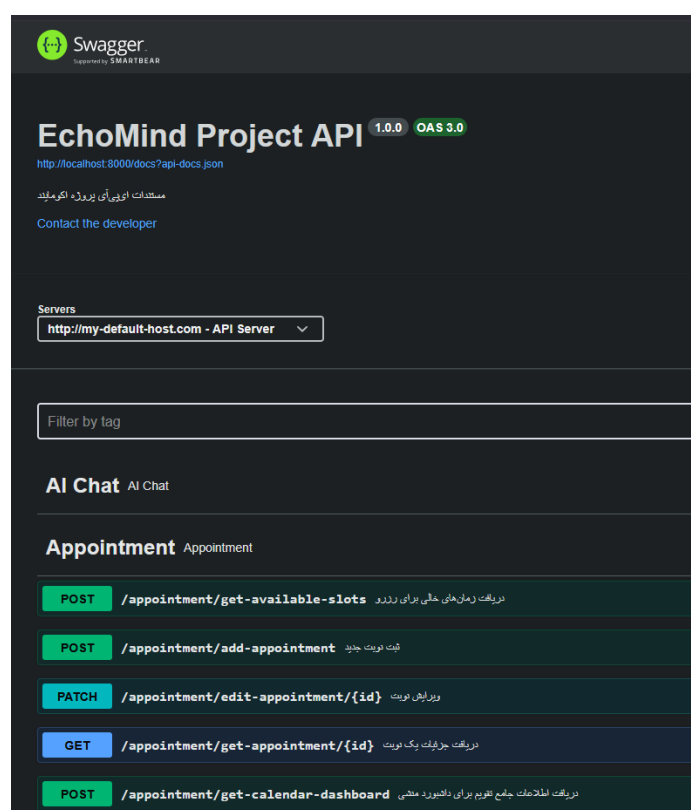
۳-۴-۵ مدیریت علائم حیاتی

کنترلر **VitalController** واسطه **API** میان فرانت‌اند و زیرسیستم پایش علائم حیاتی است و چهار عملیات اصلی را ارائه می‌دهد: شروع جلسه پایش جدید، پایان جلسه، دریافت داده‌های زنده جدید از یک نقطه زمانی مشخص به بعد، و ذخیره نهایی نتیجه جلسه در پروفایل بالینی بیمار. به‌صورت مستقل از این کنترلر، یک فرمان زمینه‌ای اختصاصی با نام **mqtt:listen** به‌صورت دائمی روی موضوع الگو

devices/+/data در بروکر Mosquitto مشترک می‌شود؛ این طراحی امکان می‌دهد سیستم بدون نیاز به تغییر کد، از چندین دستگاه ESP۳۲ مستقل به‌صورت هم‌زمان داده دریافت کند.

۳-۴-۶ مستندسازی API

مستندات کامل API با استفاده از حاشیه‌نویسی استاندارد OpenAPI/Swagger مستقیماً در کد کنترلرها تعریف شده و به‌صورت خودکار در مسیر api-docs قابل مشاهده و آزمایش تعاملی است. این رویکرد، مستندات API را همواره هم‌گام و هم‌راستا با کد واقعی نگه می‌دارد.



شکل ۸: تصویر مستندات API (Swagger)

۳-۵ پیاده‌سازی سرویس هوش مصنوعی و چت‌بات

دستیار هوشمند سیستم از ترکیب دو مؤلفه مستقل شکل گرفته است: کنترلر `AIChatController` در سمت `Laravel` که نقش هماهنگ‌کننده کلی جریان گفتگو را دارد، و سرویس مستقل `FAISS` که وظیفه تخصصی تشخیص نیت پیام و جستجوی معنایی را بر عهده دارد.

۳-۵-۱ تشخیص نیت پیام

در اولین گام پردازش هر پیام دریافتی از کاربر، `AIChatController` درخواستی به نقطه پایانی داخلی `POST /route` سرویس `FAISS` ارسال می‌کند. این نقطه پایانی از ترکیبی دومرحله‌ای برای تشخیص نیت استفاده می‌کند: ابتدا تطبیق مستقیم کلیدواژه‌های پرتکرار و بدون ابهام انجام می‌شود؛ در صورت عدم قطعیت از این روش، جستجوی برداری با استفاده از `embedding` پیام کاربر و مقایسه شباهت کسینوسی با نمونه‌های آموزشی از پیش برچسب‌گذاری‌شده، تصمیم نهایی را اتخاذ می‌کند.

۳-۵-۲ جریان رزرو نوبت گفتگومحور

در صورت تشخیص نیت رزرو نوبت، `AIChatController` با فراخوانی مدل زبانی بزرگ، اطلاعات موردنیاز رزرو را به‌صورت مرحله‌ای و از طریق چندین نوبت پیام‌رسانی با کاربر استخراج می‌کند. برای اطمینان از قابلیت پردازش برنامه‌نویسی‌شده خروجی مدل زبانی، از یک درخواست ساخت‌یافته استفاده می‌شود که مدل را ملزم به بازگرداندن اطلاعات استخراج‌شده در قالب یک شیء `JSON` می‌کند؛ این خروجی در سمت `Laravel` اعتبارسنجی و در صورت ناقص بودن هر فیلد، پرسش تکمیلی مناسب برای همان فیلد خاص به کاربر بازگردانده می‌شود. پس از تکمیل کامل اطلاعات، نوبت از طریق فراخوانی مستقیم سرویس داخلی نوبت‌دهی ثبت می‌شود.





شکل ۹: تصویر رابط گفتگو با دستیار هوشمند

۳-۵-۳ جریان پشتیبانی مبتنی بر بازیابی اطلاعات

در صورت تشخیص نیت پشتیبانی، پیام کاربر به نقطه پایانی `POST /search-faiss` سرویس FAISS ارسال می‌شود. این سرویس در گام نخست، فهرستی از پرسش‌های بسیار پرتکرار و حساس را با یک نگاشت کلیدواژه مستقیم و قطعی پاسخ می‌دهد. در صورتی که پیام کاربر با هیچ‌کدام از این کلیدواژه‌های دقیق مطابقت نداشته باشد، سیستم به جستجوی برداری متوسل می‌شود؛ نزدیک‌ترین سند تنها در صورتی که میزان شباهت آن از یک آستانه تطبیقی عبور کند، به‌عنوان پاسخ معتبر در نظر گرفته می‌شود.^{۳۲}

۳-۶ پیاده‌سازی خط پردازش^{۳۳} پایتونی اکوکاردیوگرافی

این بخش، هسته فناورانه و علمی اصلی پروژه و مسئول تحلیل کاملاً خودکار ویدیوی اکوکاردیوگرافی است. اجرای آن از طریق خط فرمان صورت می‌گیرد.

۳-۶-۱ گام به گام روند پردازش یک ویدیوی اکوکاردیوگرافی

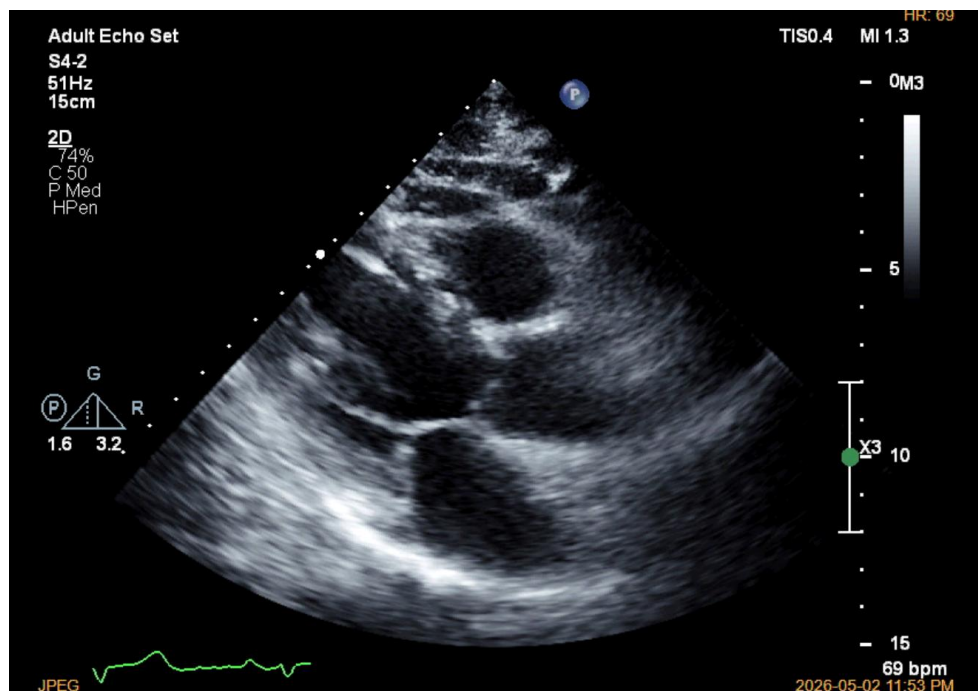
ماژول `pipeline/processing.py`، به عنوان نقطه ورود منطق پردازش تصویر، نقش یک هماهنگ‌کننده (Orchestrator) را ایفا می‌کند که به ترتیب دقیق، خروجی هر گام را به عنوان ورودی گام بعدی به کار می‌گیرد. برای روشن تر شدن این روند برای خواننده، هر گام به همراه نوع دقیق مدل یا روش به کاررفته در آن شرح داده می‌شود:

گام اول - طبقه‌بندی نما^{۳۴}

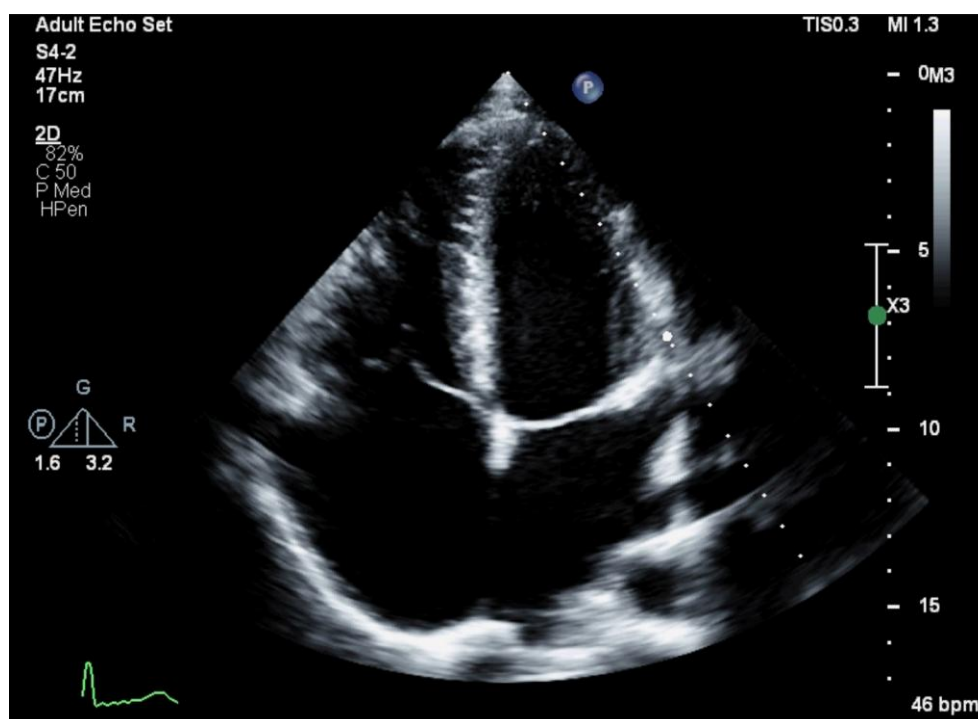
نخستین گام، تشخیص خودکار این است که ویدیوی ورودی به کدام یک از نماهای استاندارد اکوکاردیوگرافی (مانند A4C یا PLAX) تعلق دارد. این کار با استفاده از یک مدل از پیش آموزش دیده (Pre-trained) مبتنی بر Keras/TensorFlow انجام می‌شود که شامل یک شبکه کانولوشنی (CNN) برای استخراج ویژگی‌ها و لایه‌های تمام متصل (Dense) جهت طبقه‌بندی نهایی نما است. مدل آموزش دیده در قالب فایل H5 بارگذاری شده و با دقتی در حدود ۹۳٪، نمای ورودی را در یکی از کلاس‌های استاندارد اکوکاردیوگرافی قرار می‌دهد. همچنین در پیاده‌سازی، سازگاری مدل با نسخه‌های مختلف TensorFlow/Keras از طریق مدیریت تغییرات ساختاری (مانند جایگزینی SafeFlatten و تنظیم custom_objects) در نظر گرفته شده است تا مدل بدون وابستگی به نسخه کتابخانه‌ها اجرا شود. تعیین نما ضروری است زیرا هر نما نیازمند الگوریتم اندازه‌گیری اختصاصی خود در گام‌های بعدی است.

^{۳۳}. PipeLine

^{۳۴}. View Classification



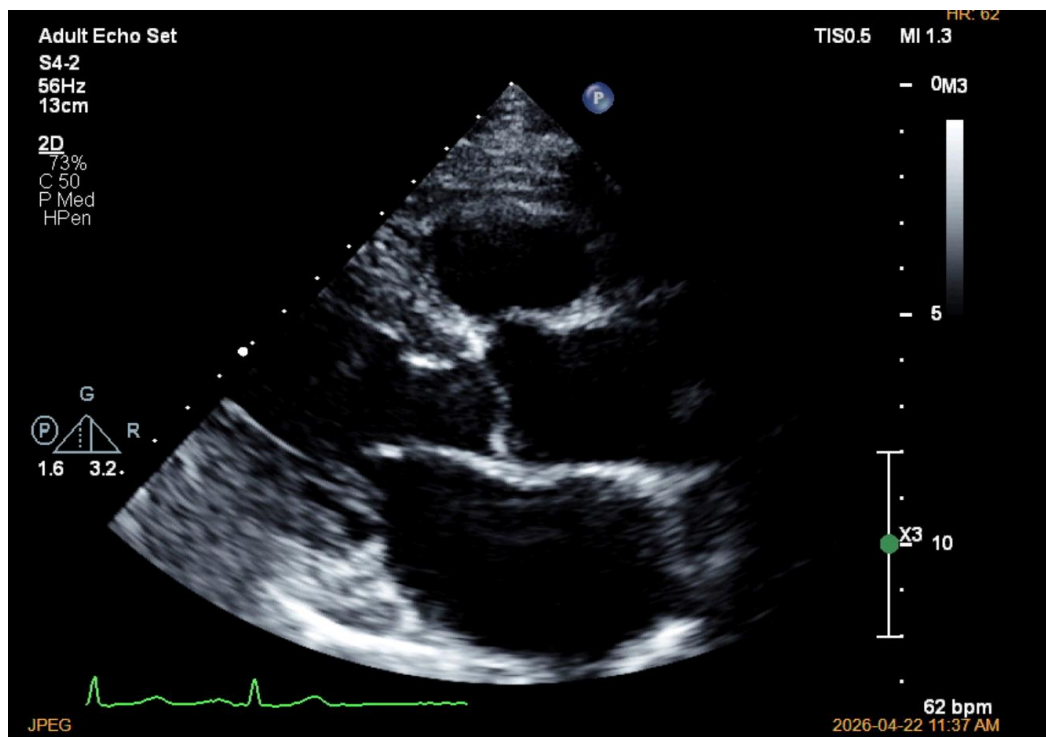
شکل ۱۰: تصویر اکوی بیمار در ناحیه PLAX



شکل ۱۱: تصویر اکوی بیمار در ناحیه A4C

گام دوم - استخراج رویداد چرخه قلبی^{۳۵}

در این مرحله، فریم‌های کلیدی چرخه قلب شامل پایان دیاستول (End Diastole)، LVOT و پایان سیستول (End Systole) به صورت خودکار از ویدیوی اکوکاردیوگرافی استخراج می‌شوند. با استفاده از تحلیل نوار ECG موجود در پایین ویدیو، لحظه‌های مهم چرخه قلب را شناسایی می‌کند. در ابتدا، ناحیه مربوط به نوار ECG و ناحیه اصلی تصویر اکو به صورت ثابت از تصویر جدا می‌شوند. سپس با تبدیل فضای رنگی تصویر به HSV و استخراج نوار سبز رنگ ECG، سیگنال قلب از روی ویدیو بازسازی می‌شود.

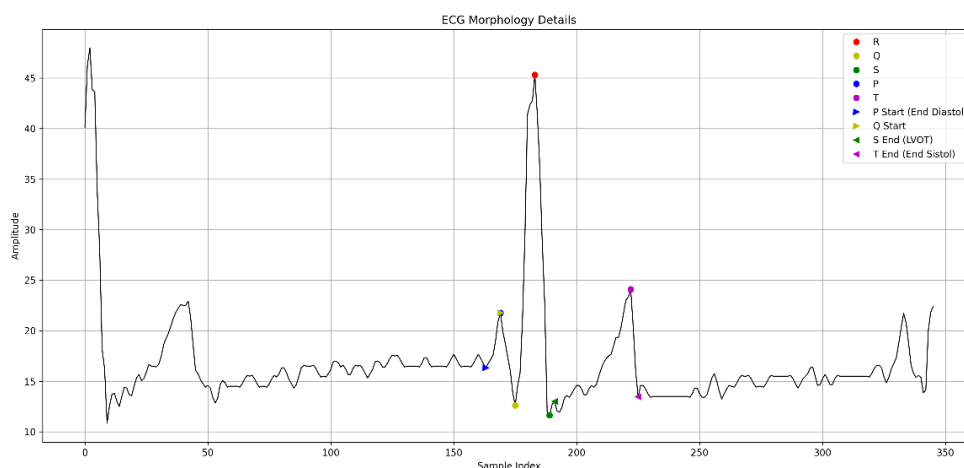


شکل ۱۲: تصویر خام اکوی بیمار همراه با نوار قلب سبز رنگ

پس از استخراج سیگنال، به منظور کاهش نویز و حفظ شکل اصلی موج، از فیلتر Savitzky-Golay برای هموارسازی استفاده می‌شود. سپس قله‌های اصلی ECG شامل P، Q، R، و S و T شناسایی شده و نقاط شروع و پایان هر موج تعیین می‌شوند. بر اساس این اطلاعات، رویدادهای مهم چرخه قلب

شامل End Diastole (شروع موج P)، LVOT (پایان موج S) و End Systole (پایان موج T) استخراج شده و آخرین چرخه کامل برای ادامه پردازش انتخاب می‌شود.

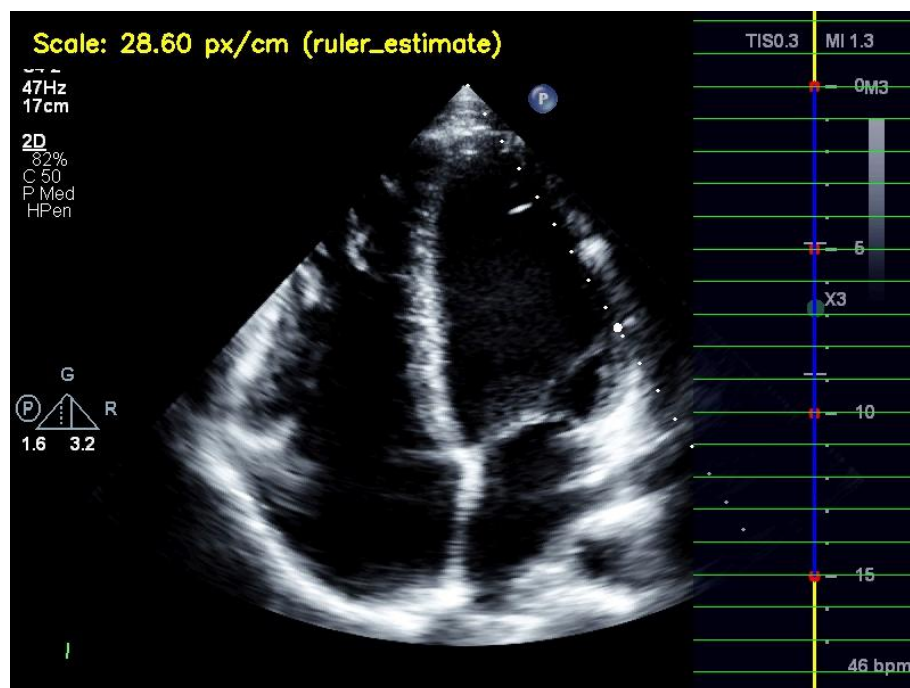
در ادامه، فریم متناظر با هر رویداد از ویدیو بازیابی، ناحیه اصلی اکو برش داده شده و تصویر نهایی در پوشه مربوط به همان رویداد ذخیره می‌شود. علاوه بر این، سیستم یک نمودار مشابه سیگنال ECG تولید می‌کند تا پزشک بتواند صحت رویدادهای استخراج‌شده را به صورت بصری بررسی و تأیید کند. خروجی این مرحله شامل شماره فریم رویدادهای مهم، تصاویر استخراج‌شده و اطلاعات موردنیاز برای انجام اندازه‌گیری‌های بعدی است.



شکل ۱۳: تصویر نمودار ECG استخراج‌شده

گام سوم - برآورد مقیاس^{۳۶}

تشخیص و پردازش خط‌کش مدرج موجود در حاشیه تصویر دستگاه سونوگرافی؛ این گام کاملاً بر پایه تکنیک‌های بینایی کامپیوتر کلاسیک (نظیر تشخیص لبه و تحلیل الگوی نقاط مدرج) پیاده‌سازی شده و از یادگیری عمیق در آن استفاده نشده است. خروجی این گام، ضریب تبدیل فاصله پیکسلی به واحد استاندارد سانتی‌متر است که در گام‌های بعدی برای تمامی اندازه‌گیری‌ها به کار می‌رود.



شکل ۱۴: تصویر برآورد مقیاس از خط‌کش مدرج

گام چهارم - اندازه‌گیری پارامترهای دایامتر

اندازه‌گیری پارامترهای خطی قلب شامل دایامتر بین‌بطنی^{۳۷}، قطر داخلی بطن چپ^{۳۸}، دیواره‌ی خلفی بطن چپ^{۳۹}، آئورت، ریشه‌ی آئورت^{۴۰} و قاعده‌ی بطن راست^{۴۱} با استفاده از یک مدل سگمنتیشن اختصاصی مبتنی بر معماری DeepLabV۳+ با بک‌بون ResNet۵۰ انجام می‌شود. برخلاف مدل سگمنتیشن ناحیه‌ای بطن (گام سگمنتیشن)، خروجی این مدل مختصات دقیق دو نقطه‌ی ابتدا و انتهای هر خط اندازه‌گیری است، نه یک ماسک ناحیه‌ای.

مدل به ازای هر پارامتر (مثلاً `rv_base`) دارای وزن‌های اختصاصی و از پیش آموزش‌دیده‌ی مجزا است. خروجی خام مدل دو کانال مستقل (نه رقیب) است که با تابع Sigmoid به دو نقشه‌ی احتمال تبدیل می‌شوند: کانال اول احتمال نقطه‌ی ابتدای خط و کانال دوم احتمال نقطه‌ی انتهای خط را در هر پیکسل نشان می‌دهد.

برای استخراج مختصات از این نقشه‌های احتمال، تابع `segmentation_to_coordinates` با روش مرکز جرم وزنی^{۴۲} به کار می‌رود؛ به این صورت که مختصات هر پیکسل با وزن احتمال (خروجی Sigmoid) آن ضرب و میانگین‌گیری وزن‌دار می‌شود تا نقطه‌ی نهایی به‌دست آید. سپس فاصله‌ی اقلیدسی دو نقطه محاسبه (طول پیکسلی) و با استفاده از تابع `length_cm_from_model_line` و مقدار کالیبره‌شده‌ی `pixels_per_cm`، طول واقعی خط بر حسب سانتی‌متر برآورد می‌شود؛ این تابع مختصات را از فضای ثابت مدل (۶۴۰×۴۸۰) به فضای ابعاد واقعی تصویر ورودی نگاشت می‌کند تا مقیاس اندازه‌گیری با ابعاد اصلی تصویر اکو سازگار باشد.

۳۷. IVS

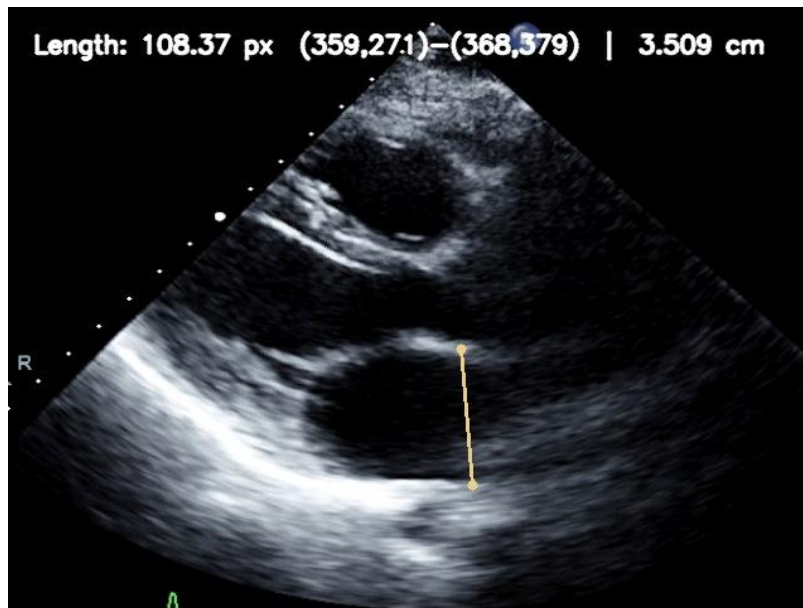
۳۸. LVID

۳۹. LVPW

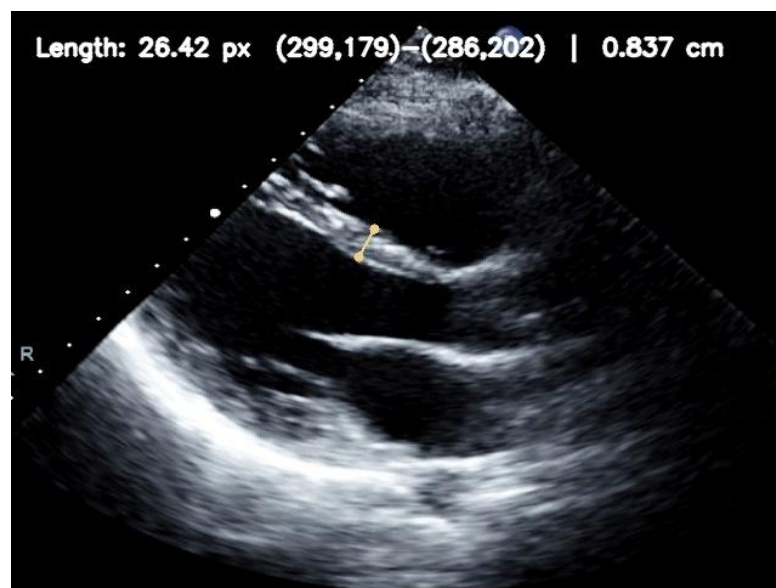
۴۰. Aortic Root

۴۱. RV Base

۴۲. Weighted Centroid



شکل ۱۵: اندازه گیری دایامتر la در لحظه End Systol



شکل ۱۶: اندازه گیری دایامتر ivs در لحظه End Diastol

گام پنجم - اندازه‌گیری دهلیز چپ با بینایی کامپیوتر کلاسیک.

برخلاف بطن چپ، اندازه‌گیری مساحت دهلیز چپ (و همچنین دهلیز راست) از هیچ مدل یادگیری عمیقی استفاده نمی‌کند و کاملاً بر پایه‌ی یک زنجیره از تکنیک‌های کلاسیک بینایی کامپیوتر انجام می‌شود. این عملیات تنها زمانی توسط `processing.process_video` فراخوانی می‌شود که نمای ویدیو به‌عنوان AEC (اپیکال چهارحفره‌ای) شناسایی شده باشد.

فرآیند پردازش در چند مرحله انجام می‌شود: ابتدا با جست‌وجوی شبکه‌ای در ناحیه‌ی مرکزی تصویر و بررسی روشنایی پرتوهایی در چهار جهت اصلی، مرکز بهینه‌ی قلب^{۴۳} شناسایی می‌شود؛ سپس با آستانه‌گذاری باینری و عملیات مورفولوژیک، دیواره‌های قلب از فضای داخلی (خون) جدا می‌شوند. در ادامه با استخراج و پالایش کانتورها و اتصال هوشمند شکاف‌های کوچک دیواره^{۴۴} ضمن حفظ یک ناحیه‌ی استثنا در اطراف دریچه‌ی دهلیزی-بطنی برای جلوگیری از اتصال اشتباه دهلیز و بطن - حلقه‌ی دیواره‌ای هر حفره کامل می‌شود. برای هر دهلیز یک نقطه‌ی `seed` (بر اساس تاریک‌ترین پیکسل در همسایگی نقطه‌ی حدسی حاصل از مرکز قلب و پرتوهای جهت‌دار) انتخاب و عملیات پرکردن سیلابی^{۴۵} از آن نقطه اجرا می‌شود تا فضای داخلی حفره مشخص گردد. در گام پایانی، یک بیضی^{۴۶} بر مرز به‌دست‌آمده `fit` شده و ناحیه‌ی حاصل موازی با صفحه‌ی دریچه‌ی دهلیزی-بطنی برش داده می‌شود تا دو دهلیز چپ و راست به‌طور دقیق از هم جدا شوند.

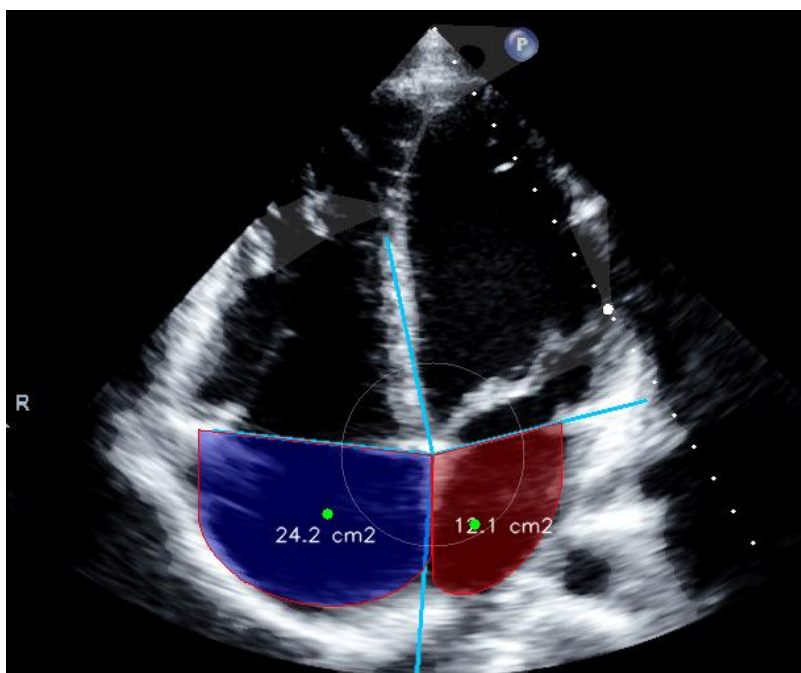
این رویکرد کلاسیک آگاهانه انتخاب شده است، زیرا مرز دهلیز چپ در نمای AEC معمولاً کنتراست و شکل هندسی به‌نسبت منظم‌تری نسبت به بطن دارد که امکان اندازه‌گیری با روش‌های کلاسیک بینایی کامپیوتر و بدون نیاز به آموزش یک مدل یادگیری عمیق اختصاصی را فراهم می‌کند.

۴۳. `best_center`

۴۴. `Bridging`

۴۵. `Flood Fill`

۴۶. `Ellipse`



شکل ۱۷: تشخیص و اندازه‌گیری دهلیز با بینایی کامپیوتر کلاسیک

گام ششم - قطعه‌بندی معنایی و محاسبه حجم با مدل آموزش داده‌شده.

برای استخراج دقیق مرز کامل حفرات قلب (به‌ویژه بطن چپ) و محاسبه‌ی ابعاد، مساحت و حجم آنها (از جمله در فریم‌های پایان دیاستول و پایان سیستول)، از یک مدل قطعه‌بندی معنایی^{۴۷} با معماری U-Net++ و رمزگذار^{۴۸} از نوع timm-efficientnet-b۴ استفاده شده است.

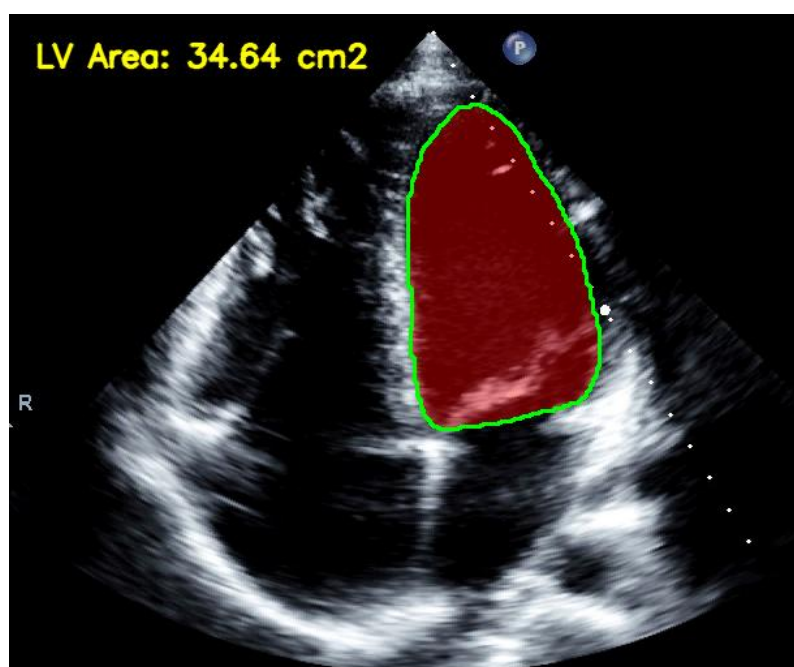
داده‌ی آموزشی از مجموعه‌داده‌ی عمومی و تخصصی EchoNet-Dynamic استخراج شده که شامل فریم‌های اکوکاردیوگرافی به‌همراه مرزبندی دستی متخصص (فایل VolumeTracings.csv) است؛ این مرزبندی‌ها به فریم و ماسک دودویی تبدیل شده‌اند. آموزش با تابع خطای ترکیبی Dice Loss و Binary Cross-Entropy، بهینه‌ساز AdamW، زمان‌بند نرخ یادگیری ReduceLROnPlateau، تکنیک‌های تقویت داده (Horizontal Flip، Elastic Transform، Grid Distortion و...) برای

^{۴۷}. Semantic Segmentation

^{۴۸}. Encoder

افزایش تعمیم‌پذیری مدل به تنوع بصری واقعی تصاویر، و همچنین راهبردهای Early Stopping و Mixed Precision Training انجام شده است. فرآیند آموزش دارای قابلیت ذخیره و بازیابی خودکار نقطه بازرسی^{۴۹} نیز بوده تا امکان ادامه‌ی آموزش پس از قطع‌شدن جلسه‌ی محاسباتی به دلایل شبکه‌ای (مانند محیط Google Colab) وجود داشته باشد.

وزن نهایی مدل با بهترین معیار Dice در فایل best.pth ذخیره شده و برای استنتاج به کار گرفته می‌شود. خروجی این مدل، ماسک دودویی بطن چپ در هر فریم است که مبنای محاسبات بعدی ابعاد و حجم قرار می‌گیرد.

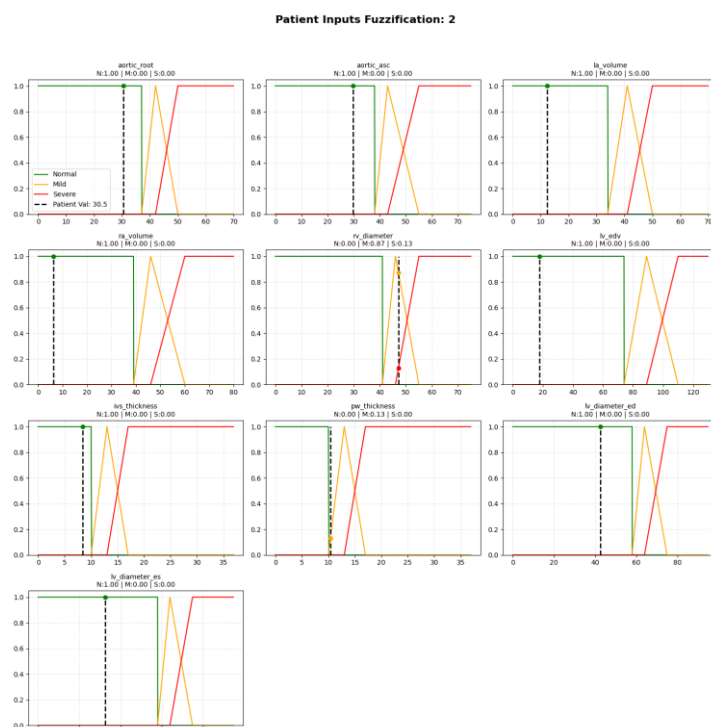


شکل ۱۸: تشخیص و اندازه‌گیری بطن چپ با مدل آموزش داده شده

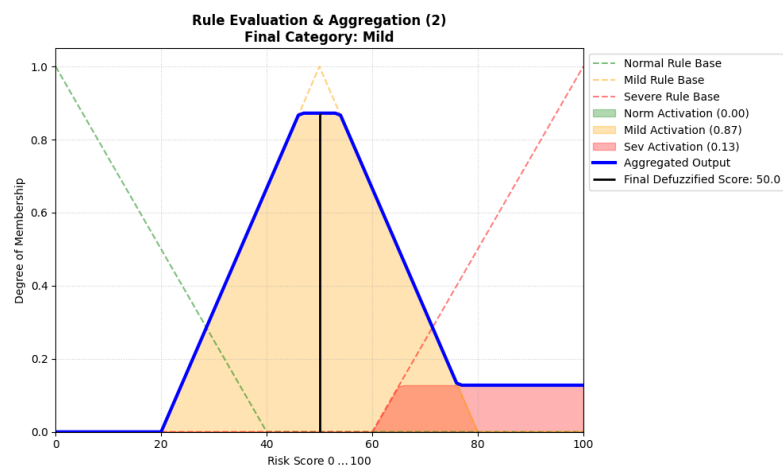
^{۴۹}. Checkpoint

گام هفتم - تجميع فازي

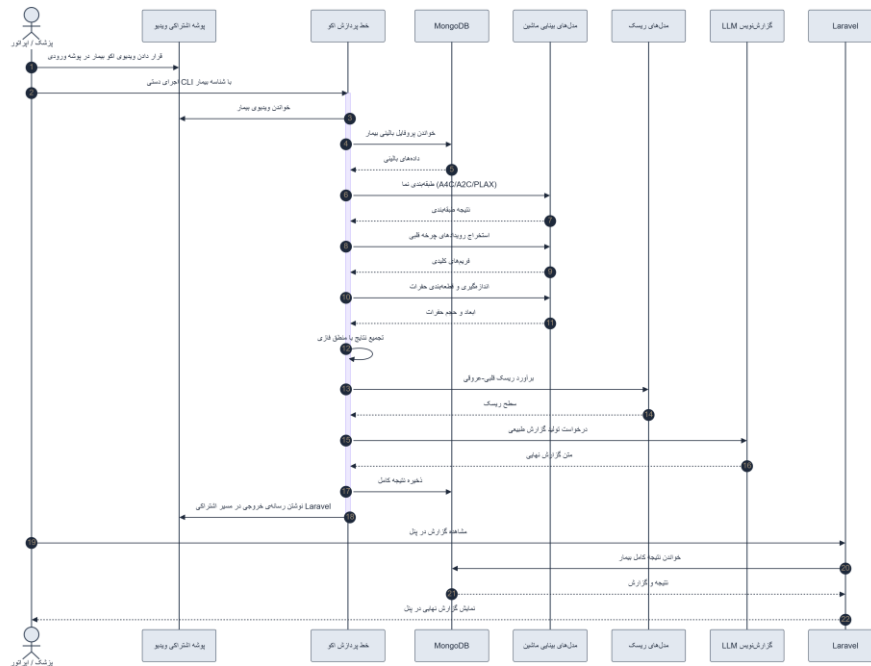
در گام پاياني تحليل تصوير و تمامي نتايج عددي حاصل از گام‌هاي قبل (اندازه‌گيري دايمتر، بطن، دهليز و مسير خروجي بطن) توسط لايه منطق فازي پياده‌سازي‌شده، با يکديگر ترکيب و به يک ارزيابي کيفي و کمي نهايي و قابل فهم براي پزشک تبديل مي‌شوند.



شکل ۱۹: نمودارهاي ورودی سیستم فازي



شکل ۲۰: نمودار خروجي تجميع شده سیستم فازي



شکل ۲۱: نمودار توالی - خط پردازش اکوکاردیوگرافی

۳-۶-۲ برآورد ریسک قلبی - عروقی با دو مدل یادگیری ماشین آموزش داده شده

ماژول `ai_service/ml_predictor.py` داده‌های بالینی بیمار را که پیش‌تر توسط کاربر در فرانت‌اند وارد و در پایگاه داده MongoDB ذخیره شده است، فراخوانی می‌کند. این داده‌ها شامل ویژگی‌های بالینی استاندارد مانند سن، جنسیت، فشار خون سیستولیک و دیاستولیک، سطح کلسترول، ضربان قلب حداکثر و سایر شاخص‌های بالینی است.

شکل ۲۲: اطلاعات برآورد ریسک قلبی - عروقی کاربر

بر پایه‌ی این اطلاعات، ماژول دو نوع ریسک مستقل را به صورت کمی برآورد می‌کند.

هر دو مدل ریسک قلبی این بخش آموزش داده شده‌اند. فرآیند کامل آموزش، ارزیابی و تفسیرپذیری این مدل‌ها در دفترچه‌ی Jupyter مستند و قابل بازتولید است. این دفترچه دو مجموعه داده بالینی مستقل را پردازش می‌کند: مجموعه داده Heart Disease (بر پایه‌ی مجموعه داده‌ی شناخته شده‌ی UCI) برای برآورد ریسک بیماری قلبی، و مجموعه داده Cardiovascular برای برآورد ریسک قلبی-عروقی. مراحل زیر برای هر دو مجموعه داده به طور کامل طی شده است:

۱. تحلیل اکتشافی داده^{۵۰}

هدف: پیش از انجام هرگونه پردازش یا مدل سازی، ابتدا ساختار و کیفیت دو دیتاست بررسی شد تا تصمیمات بعدی پروژه (نحوه‌ی پاک سازی، انتخاب فیچر، و مدیریت عدم تعادل کلاس‌ها) بر پایه‌ی شناخت واقعی از داده اتخاذ شود.

اقدامات انجام شده: بررسی ابعاد، نوع داده‌ها^{۵۱} و تعداد مقادیر گمشده در هر ستون، بررسی تعداد رکوردهای تکراری در هر دو دیتاست، رسم نمودار توازن کلاس‌ها^{۵۲} برای برچسب هدف، رسم توزیع سن به تفکیک کلاس بیمار/سالم، رسم نقشه حرارتی همبستگی^{۵۳} بین ویژگی‌های عددی



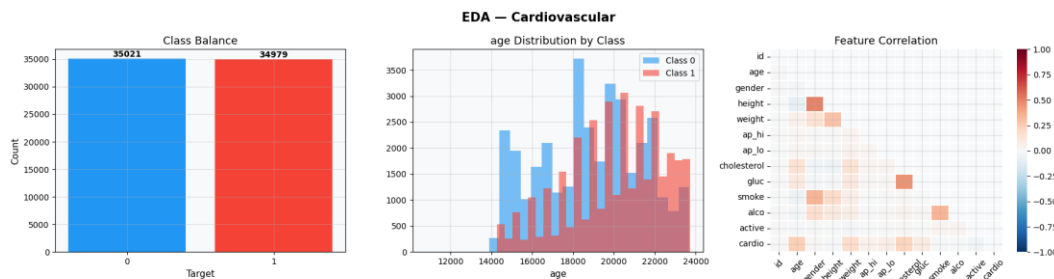
شکل ۲۳: خروجی تحلیل اکتشافی دیتاست Heart Disease

۵۰. EDA

۵۱. dtype

۵۲. Class Balance

۵۳. Correlation Heatmap



شکل ۲۴: خروجی تحلیل اکتشافی دیتاست Cardiovascular

تحلیل نتایج: بررسی توزیع کلاس‌ها نشان داد هر دو دیتاست تعادل نسبی قابل قبولی بین کلاس بیمار و سالم دارند، بنابراین نیازی به تکنیک‌های خاص مقابله با عدم توازن شدید (مانند SMOTE) نبود. توزیع سن نیز نشان داد بیماران مبتلا به طور میانگین در سنین بالاتری نسبت به افراد سالم قرار دارند که با شواهد بالینی شناخته شده هم‌راستا است. نقشه‌ی همبستگی نیز چند ویژگی (از جمله oldpeak، cp و thalach در دیتاست Heart Disease) را به عنوان ویژگی‌های با ارتباط قوی‌تر به برچسب هدف مشخص کرد که در مرحله‌ی انتخاب و مهندسی ویژگی مورد توجه قرار گرفتند.

۲. پاک‌سازی داده پزشکی

هدف: با توجه به ماهیت پزشکی داده‌ها، حتی درصد کمی داده‌ی نامعتبر یا نویزی می‌تواند دقت مدل را به طور محسوسی کاهش دهد؛ به همین دلیل یک مرحله پاک‌سازی ساختاریافته و چندمرحله‌ای برای هر دو دیتاست طراحی و اجرا شد.

اقدامات انجام شده: حذف رکوردهای کاملاً تکراری، تبدیل واحد سن از روز به سال در دیتاست Cardiovascular (که در فایل خام بر حسب روز ثبت شده بود)، حذف مقادیر غیرممکن فیزیولوژیک؛ از جمله سن خارج از بازه‌ی ۱۸ تا ۱۲۰ سال، فشار خون سیستولی/دیاستولی خارج از بازه‌ی معتبر، مواردی که فشار دیاستولی بزرگ‌تر یا مساوی فشار سیستولی بود، کلسترول خارج از بازه‌ی ۱۰۰ تا ۶۰۰، ضربان قلب خارج از بازه‌ی ۴۰ تا ۲۲۰، و قد/وزن خارج از بازه‌ی معقول بالینی، حذف داده‌های پرت آماری با روش IQR با ضریب محافظه‌کارانه‌ی ۳٫۵ (برای حفظ موارد نادر ولی واقعی بالینی، مثلاً حذف سن ۲۲۰)، جای‌گذاری^{۵۴} مقادیر گمشده باقی‌مانده با میانه‌ی ستون‌های عددی و مد ستون‌های متنی، و حذف

۵۴. Imputation

رکوردهایی که بیش از ۳۰٪ مقادیرشان خالی بود، تبدیل ستون‌های متنی/دسته‌ای به عددی با LabelEncoder

۳. مهندسی ویژگی پزشکی^{۵۵}

هدف: ویژگی‌های خام موجود در دیتاست‌ها به‌تنهایی سیگنال کافی برای تفکیک دقیق دو کلاس فراهم نمی‌کردند؛ بنابراین بر اساس فرمول‌های استاندارد بالینی، مجموعه‌ای از ویژگی‌های ترکیبی جدید ساخته شد (مانند شاخص توده‌ی بدنی از قد و وزن) تا اطلاعات نهفته در تعامل بین ویژگی‌های خام در اختیار مدل قرار گیرد.

اقدامات انجام‌شده: محاسبه‌ی BMI و دسته‌بندی بر اساس استاندارد (دیتاست Cardiovascular)، محاسبه‌ی میانگین فشار شریانی^{۵۶} و فشار نبض^{۵۷} از فشار سیستولی و دیاستولی، محاسبه‌ی درجه‌ی پرفشاری خون^{۵۸} و دسته‌بندی فشار خون استراحت^{۵۹} در دیتاست Heart Diseases، محاسبه‌ی شاخص‌های قلبی Rate-Pressure Product (بار کاری قلب)، Chronotropic Index (توانایی قلب به فعالیت بدنی) و شاخص افت ST (از شاخص‌های ایسکمی قلبی)، ساخت امتیاز ترکیبی ریسک متابولیک^{۶۰} و ریسک سبک زندگی^{۶۱} در دیتاست Cardiovascula، ساخت ویژگی‌های تعاملی سن با BMI، درجه‌ی پرفشاری خون و افت ST.

^{۵۵}. Feature Engineering

^{۵۶}. MAP

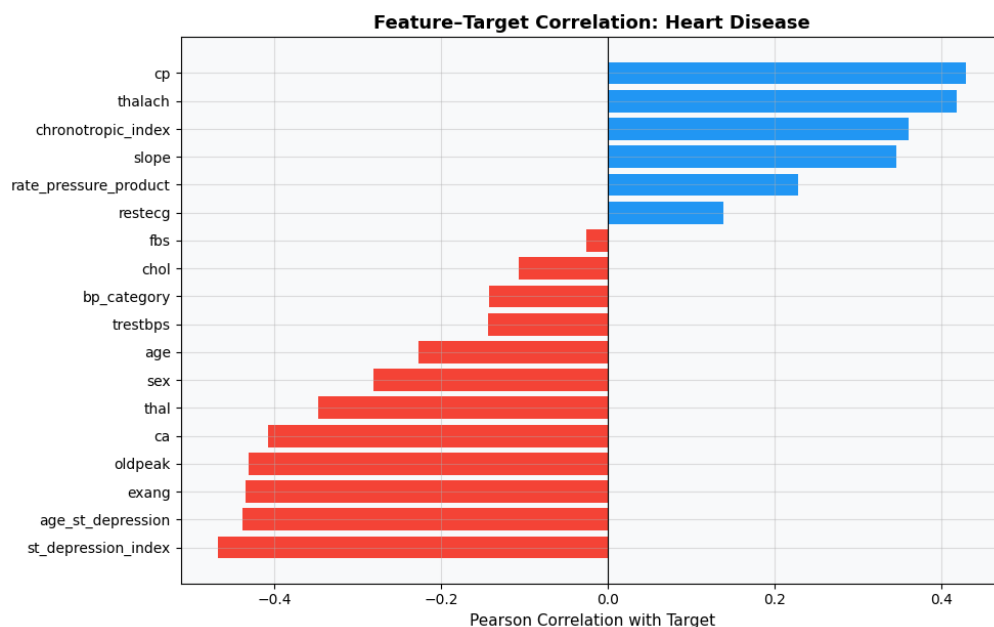
^{۵۷}. Pulse Pressure

^{۵۸}. Hypertension Grade

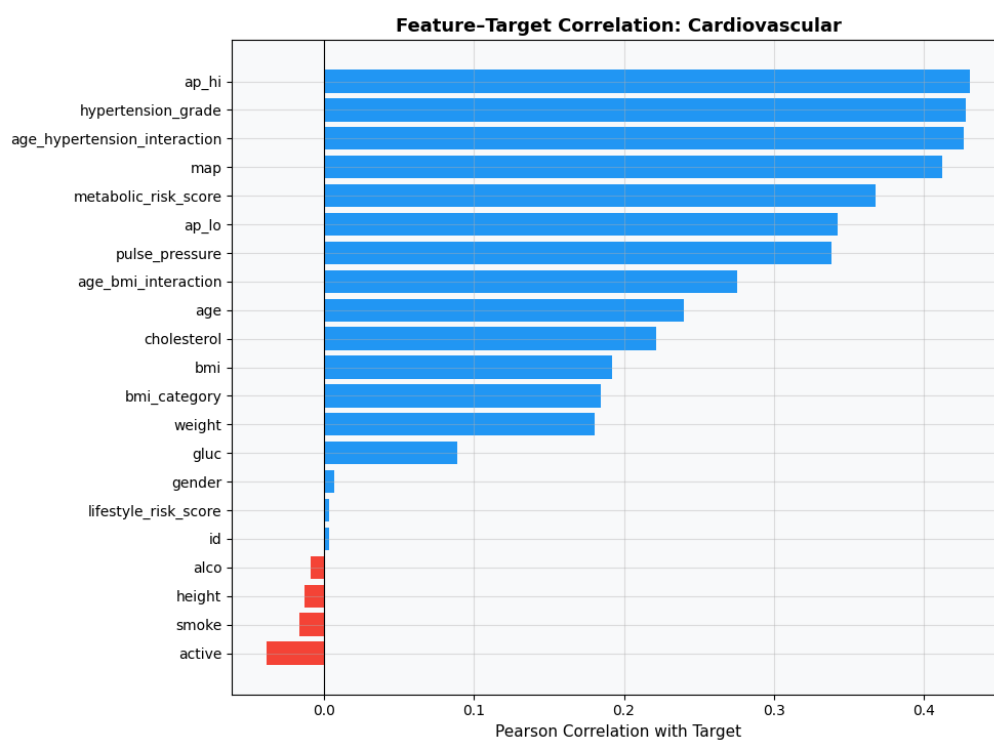
^{۵۹}. bp_category

^{۶۰}. Metabolic Risk Score

^{۶۱}. Lifestyle Risk Score



شکل ۲۵: همبستگی ویژگی‌های نهایی با برچسب هدف در دیتاست Heart Disease



شکل ۲۶: همبستگی ویژگی‌های نهایی با برچسب هدف در دیتاست Cardiovascular

تحلیل نتایج: بررسی همبستگی ویژگی‌های جدید با برچسب هدف نشان داد برخی از ویژگی‌های مهندسی شده (به‌ویژه $\text{Rate-Pressure Product}$ ، $\text{ST-Depression Index}$ و $\text{Hypertension Grade}$) همبستگی محسوسی بالاتر از ویژگی‌های خام متناظر خود داشتند؛ این موضوع فرضیه‌ی اولیه ما را تأیید کرد و این ویژگی‌ها در مجموعه‌ی نهایی ورودی مدل‌ها حفظ شدند. لازم به ذکر است چون همبستگی^{۶۲} تنها روابط خطی را نشان می‌دهد و روابطی که مدل‌های درختی مانند XGBoost قادر به یادگیری آن‌ها هستند لزوماً خطی نیستند، هیچ ویژگی‌ای بر اساس همبستگی پایین حذف نشد.

۴. آموزش چند مدل مستقل

هدف: برای اطمینان از انتخاب مناسب‌ترین الگوریتم برای هر دیتاست، به‌جای اکتفا به یک مدل، چند الگوریتم متفاوت به‌صورت هم‌زمان آموزش داده و تحت شرایط یکسان مقایسه شدند.

اقدامات انجام‌شده: تقسیم داده به مجموعه‌ی آموزش و آزمون (۸۰٪ به ۲۰٪) با حفظ نسبت کلاس‌ها^{۶۳}، آموزش چهار مدل XGBoost ، CatBoost ، LightGBM و رگرسیون لجستیک^{۶۴}، اعتبارسنجی متقاطع پنج‌لایه^{۶۵} برای هر مدل جهت برآورد پایدارتر عملکرد، استفاده از وزن‌دهی کلاس ($\text{scale_pos_weight} / \text{class_weight}$) برای کنترل عدم توازن باقی‌مانده، استانداردسازی ویژگی‌ها^{۶۶} به‌طور اختصاصی برای مدل رگرسیون لجستیک، ارزیابی نهایی هر مدل با معیارهای AUC ، $F1$ ، Precision (درصد تشخیص درست بیماران) و Recall (میزان صحت شناسایی بیماران واقعی)

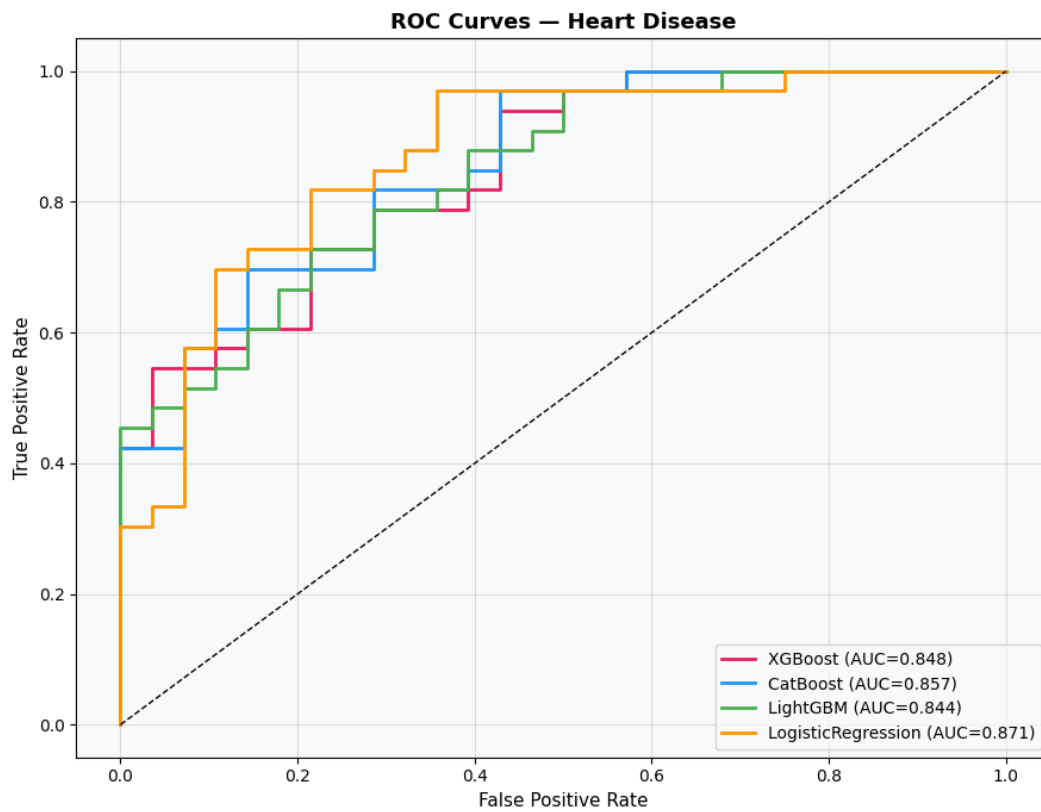
^{۶۲}. Correlation

^{۶۳}. Stratified Split

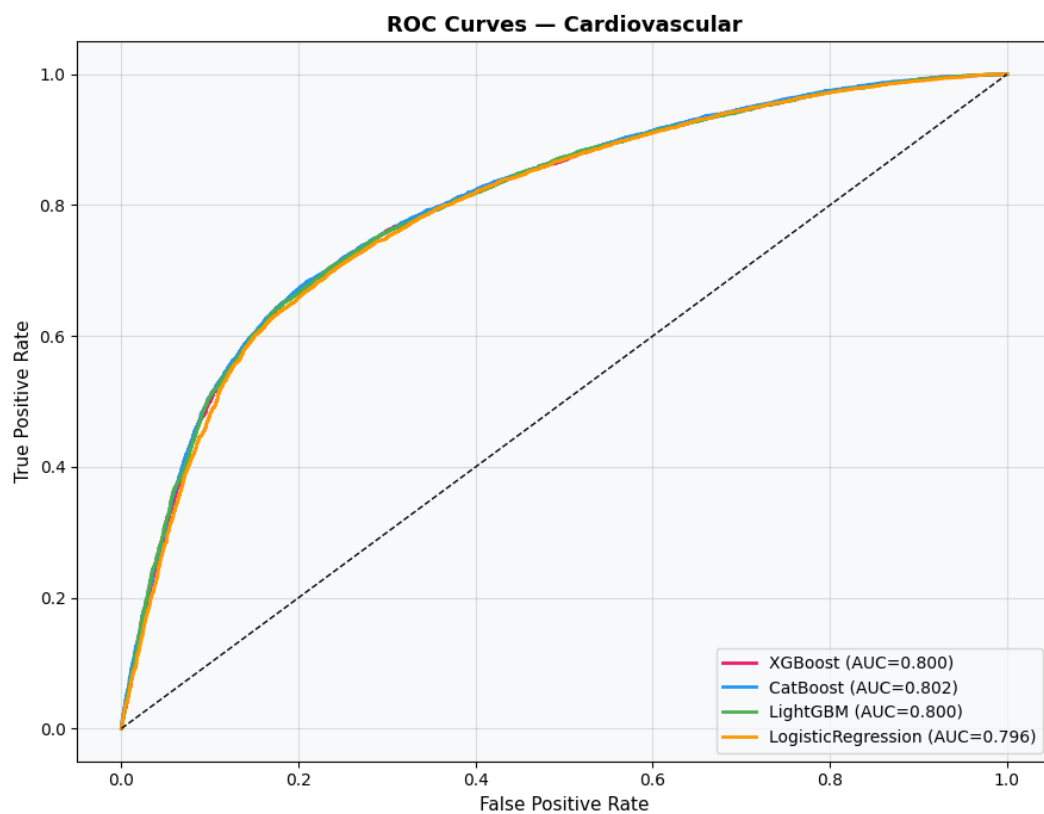
^{۶۴}. Logistic Regression

^{۶۵}. 5-Fold Stratified Cross-Validation

^{۶۶}. StandardScaler



شکل ۲۷: منحنی‌های ROC مدل‌های آموزش دیده روی دیتاست Heart Disease



شکل ۲۸: منحنی‌های ROC مدل‌های آموزش دیده روی دیتاست Cardiovascular

مساحت زیر منحنی ROC برای ارزیابی عملکرد مدل ما می باشد که مقدار بزرگ تر آن نشان دهنده قدرت تفکیک بهتر مدل است، منحنی ROC بر اساس دو شاخص نرخ مثبت کاذب و نرخ مثبت واقعی ترسیم می شود. (نرخ مثبت کاذب نسبت نمونه های سالمی را نشان می دهد که به اشتباه بیمار تشخیص داده شده اند و نرخ مثبت واقعی بیانگر نسبت نمونه های بیمار است که به درستی شناسایی شده اند)

جدول ۳: مقایسه ی عملکرد مدل های پایه

مدل	AUC (HD)	F1 (HD)	AUC (CV)	F1 (CV)
XGBoost	۰.۸۴۸۵	۰.۷۵۷۶	۰.۷۹۹۵	۰.۷۱۸۰
CatBoost	۰.۸۵۷۱	۰.۷۷۱۴	۰.۸۰۲۲	۰.۷۲۰۱
LightGBM	۰.۸۴۴۲	۰.۷۳۸۵	۰.۸۰۰۰	۰.۷۱۸۵
Logistic Regression	۰.۸۷۱۲	۰.۸۰۰۰	۰.۷۹۵۷	۰.۷۱۸۴

تحلیل نتایج: بر خلاف انتظار اولیه، مدل ساده تر Logistic Regression روی دیتاست Heart Disease بهترین عملکرد را با AUC برابر ۰.۸۷۱۲ ثبت کرد، در حالی که روی دیتاست بزرگ تر Cardiovascular، مدل CatBoost با AUC برابر ۰.۸۰۲۲ برتری داشت. دیتاست Heart Disease کوچک (حدود ۳۰۰ رکورد) است، جایی که مدل های خطی ساده تر کمتر دچار بیش برازش^{۶۷} می شوند؛ در مقابل، دیتاست Cardiovascular با حجم بسیار بیشتر امکان یادگیری الگوهای غیرخطی پیچیده تر را برای مدل های مبتنی بر درخت فراهم می کند. همین یافته مبنای انتخاب مدل نهایی متفاوت برای هر دیتاست در مرحله ی استقرار قرار گرفت.

^{۶۷} Overfitting

۵. ترکیب مدل‌ها^{۶۸}

هدف: پس از آموزش مدل‌های پایه، بررسی شد که آیا ترکیب خروجی چند مدل می‌تواند عملکرد را نسبت به بهترین مدل منفرد بهبود دهد یا خیر.

اقدامات انجام‌شده: پیاده‌سازی روش رأی‌گیری نرم^{۶۹} (میانگین‌گیری از احتمالات خروجی چهار مدل پایه)، پیاده‌سازی روش پشته‌سازی^{۷۰} با یادگیرنده‌ی متای رگرسیون لجستیک روی خروجی مدل‌های XGBoost، CatBoost و LightGBM (خروجی این مدل‌ها به‌عنوان ورودی به مدل LR داده می‌شود)، و رسم ماتریس درهم‌ریختگی^{۷۱} برای هر دو روش انسمبل

جدول ۴: مقایسه‌ی روش‌های انسمبل

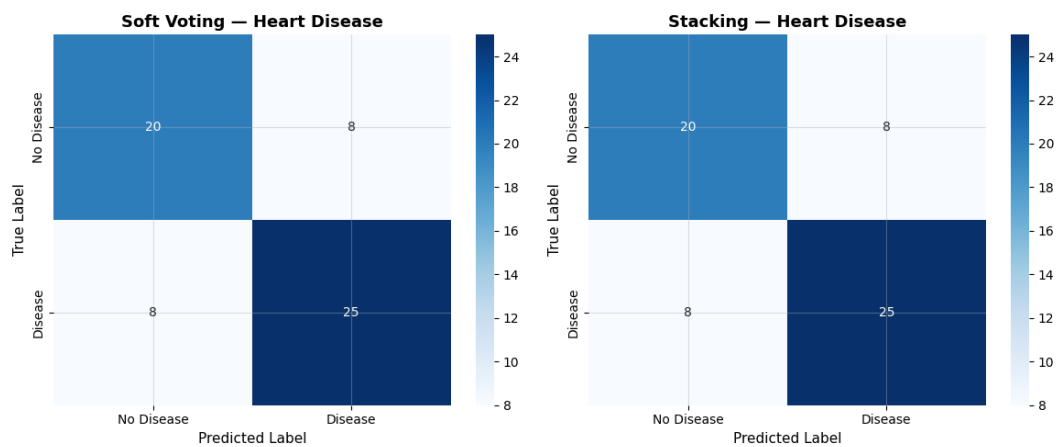
AUC (Cardiovascular)	AUC (Heart Disease)	روش انسمبل
۰.۸۰۱۳	۰.۸۵۹۳	Soft Voting
۰.۷۸۳۳	۰.۸۵۸۲	Stacking

^{۶۸}. Ensembling

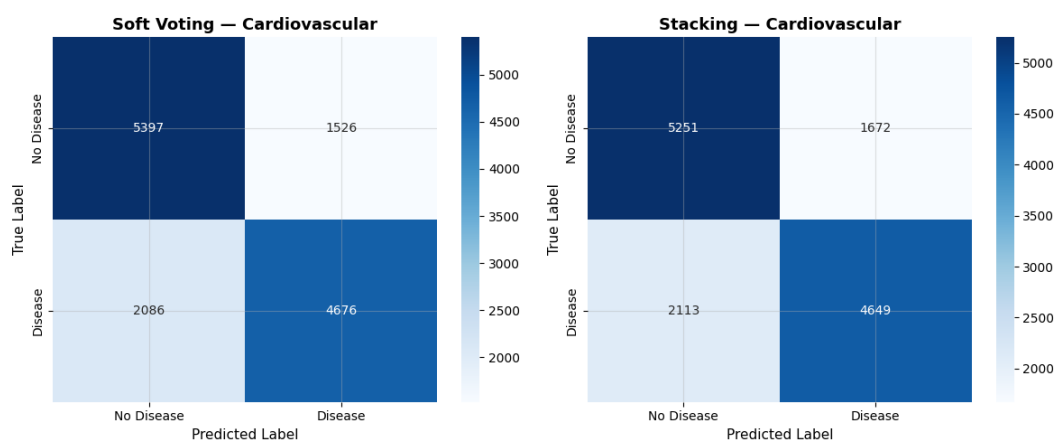
^{۶۹}. Soft Voting

^{۷۰}. Stacking

^{۷۱}. Confusion Matrix



شکل ۲۹: ماتریس درهم‌ریختگی روش‌های انسمبل روی دیتاست Heart Disease



شکل ۳۰: ماتریس درهم‌ریختگی روش‌های انسمبل روی دیتاست Cardiovascular

تحلیل نتایج: روش Soft Voting در هر دو دیتاست عملکرد بهتری نسبت به Stacking نشان داد. با این حال، بهبود حاصل از انسمبل نسبت به بهترین مدل منفرد محدود بود؛ به همین دلیل در مرحله‌ی استقرار نهایی، به جای انسمبل، مدل‌های منفرد برتر هر دیتاست انتخاب شدند تا پیچیدگی سرویس در محیط عملیاتی کاهش یابد.

۶. انتخاب مدل نهایی و اتصال به سرویس بک‌اند پروژه

هدف: پس از مقایسه‌ی مدل‌ها، لازم بود بهترین مدل هر دیتاست از محیط نوت‌بوک خارج شده و در سرویس واقعی پروژه (بخش پایتون echo) مورد استفاده قرار می‌گیرد.

اقدامات انجام‌شده: انتخاب مدل رگرسیون لجستیک برای دیتاست Heart Disease و مدل CatBoost برای دیتاست Cardiovascular بر اساس بالاترین AUC هر دیتاست.

با فراخوانی مدل زبانی بزرگ، نتایج عددی خام و ارزیابی فازی را به یک گزارش متنی روان و قابل فهم برای مطالعه پزشک یا بیمار تبدیل می‌کند. در پرامپت داده شده به مدل سعی شده خروجی قابل فهم و واضحی به بیمار داده شود. خروجی نهایی این فرآیند در نهایت اختیار فرانت‌اند قرار می‌گیرد تا بیمار با مراجعه به داشبورد خود با هر سطح عملی‌ای بتواند از وضعیت خود با خبر شود.

نمونه پیام برگشتی از مدل زبانی با توجه به وضعیت بیمار: با سلام و احترام. با بررسی نتایج اکوکاردیوگرافی شما، در مجموع وضعیت قلب شما در سطح قابل قبولی است و تغییرات شدیدی دیده نمی‌شود. با این حال، دو مورد جزئی در اندازه قلب شما مشاهده شده است؛ یکی کمی بزرگ‌تر شدن اندازه بطن راست و دیگری افزایش بسیار اندک در ضخامت دیواره قلب که در این سن نیاز به بررسی دقیق‌تر دارد تا علت آن مشخص شود. با توجه به اینکه شما در کنار این نتایج، درد قفسه سینه داشتید و بررسیات نشان‌دهنده کاهش جریان خون در برخی رگ‌های قلبی است، این موارد اهمیت بیشتری پیدا می‌کنند. در واقع، این تغییرات کوچک در ساختار قلب می‌تواند پاسخی به فشار یا کم‌خونی در عضلات قلب باشد، بنابراین نباید نادیده گرفته شوند. توصیه می‌کنم در اولین فرصت برای بررسی دقیق‌تر رگ‌های قلبی و علت این دردها، به متخصص قلب مراجعه کنید تا در صورت نیاز آزمایشات تکمیلی یا تست‌های استرسی انجام شود. همچنین تا زمان ملاقات با پزشک، از فعالیت‌های بدنی بسیار شدید پرهیز کنید و هرگونه تغییر در الگوی درد قفسه سینه را سریعاً گزارش دهید. پیگیری منظم در مورد این نتایج بسیار مهم است تا با اقدامات پیشگیرانه و اصلاح سبک زندگی، از پیشرفت هرگونه مشکل احتمالی در آینده جلوگیری شود.

۷-۳ پیاده سازی فرانت اند

فرانت اند سیستم در قالب یک اپلیکیشن مبتنی بر Angular با ساختار ماژولار و بر پایه تفکیک بر اساس

نقش کاربری پیاده سازی شده است. ساختار پوشه بندی اصلی پروژه به شرح زیر است:

shared: شامل عناصر مشترک مورد استفاده در سراسر برنامه، از جمله نگهبان های مسیریابی،

interceptor های HTTP، و سرویس های عمومی مانند ذخیره سازی محلی و ترجمه چندزبانه.

auth: کامپوننت صفحه ورود و مجموعه سرویس های مرتبط با فرآیند احراز هویت و مدیریت

اعتبارنامه های کاربر.

index: سایت عمومی و معرفی کلینیک که پیش از ورود کاربر قابل مشاهده است و شامل صفحه اصلی،

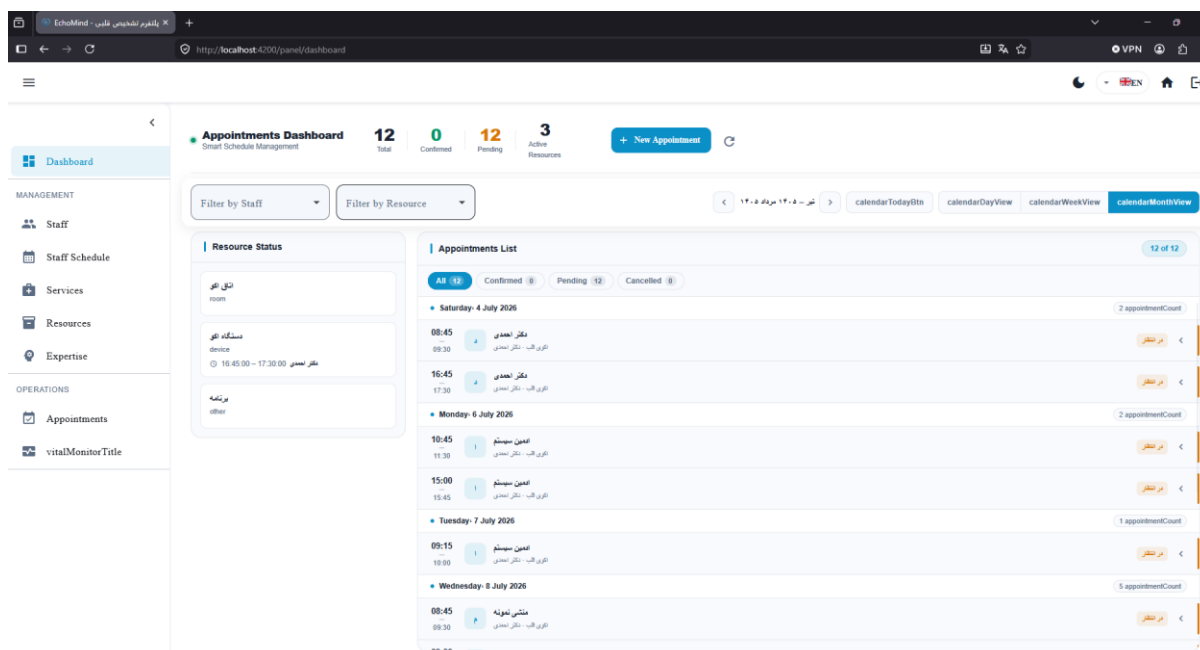
صفحه تماس، فرم رزرو نوبت عمومی، و نمایش تعاملی سه بعدی مدل قلب است.

doctor-panel: پنل اختصاصی و مجزای پزشک، شامل داشبورد شخصی، فهرست و تقویم نوبت ها،

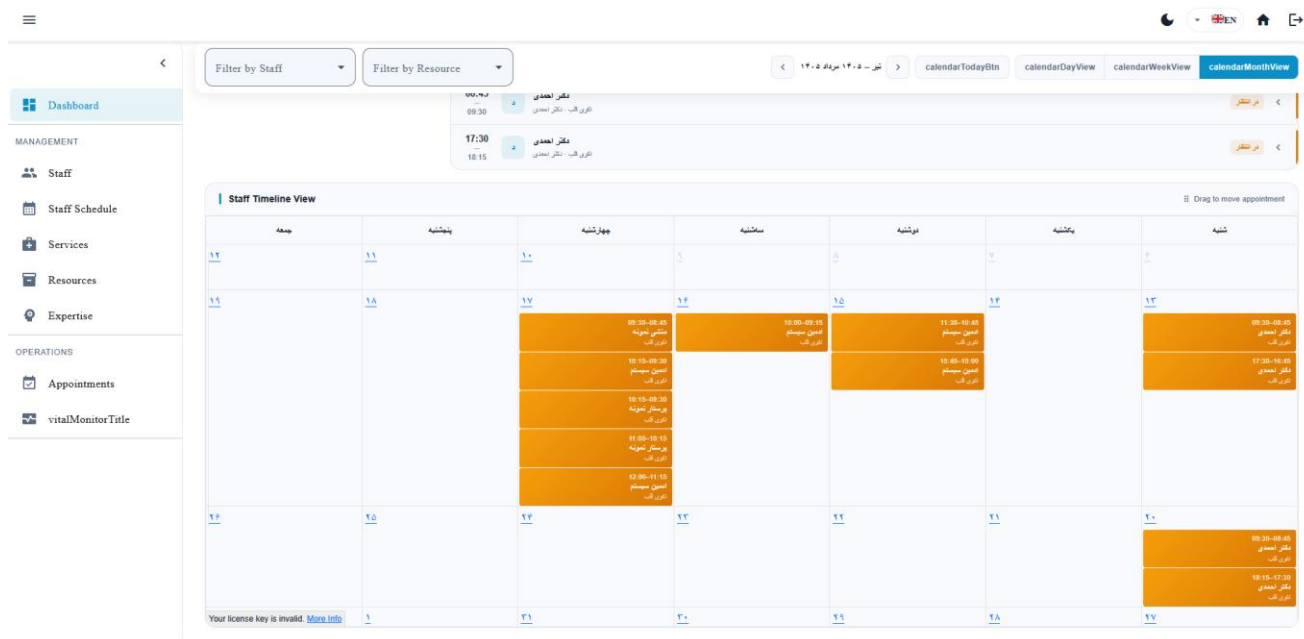
بخش تاریخچه اکو، و تنظیمات پروفایل.

panel: پنل اصلی مدیریتی و پرسنلی؛ شامل داشبورد کلی، مدیریت پرسنل و برنامه کاری، مدیریت منابع،

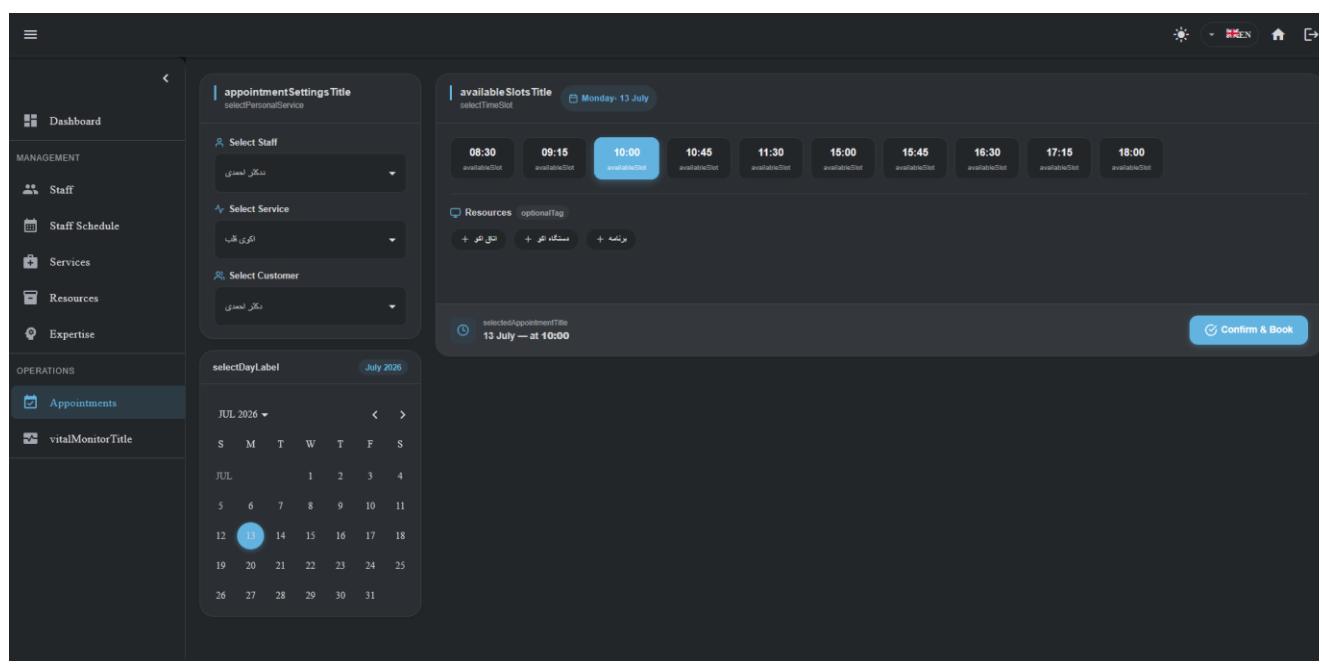
دستیار هوشمند، و پایش زنده علائم حیاتی.



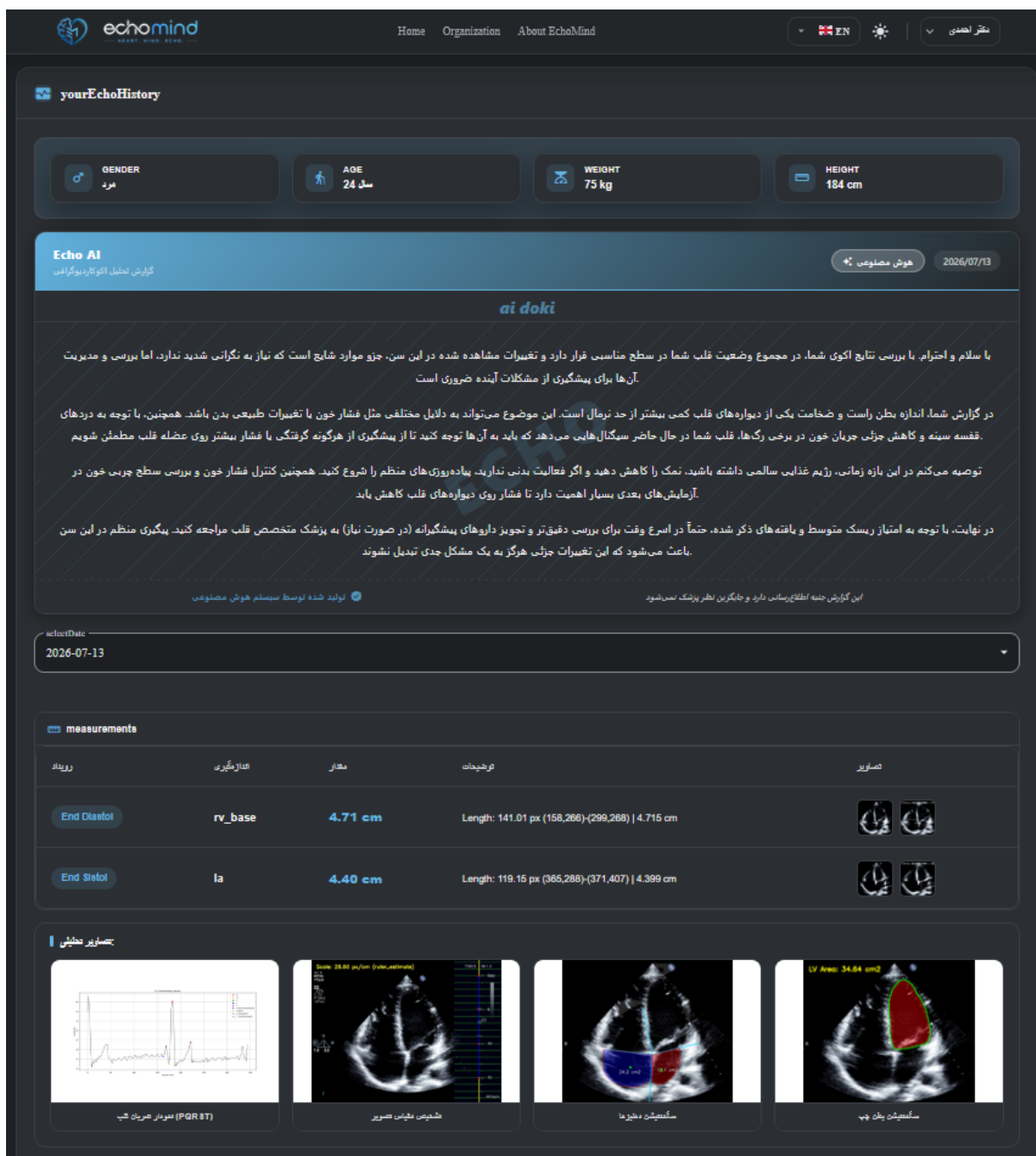
شکل ۳۱: تصویر داشبورد پنل مدیریت



شکل ۳۲: تصویر تقویم نوبت‌دهی و برنامه کاری پرسنل



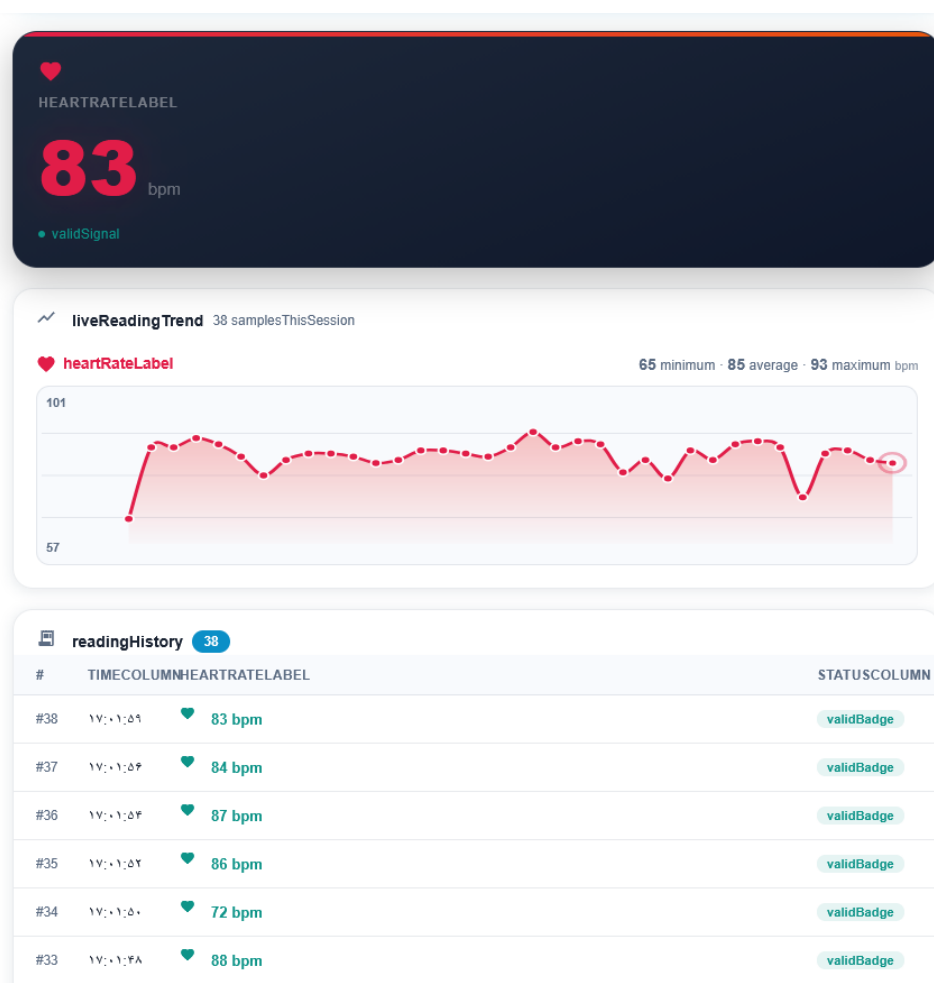
شکل ۳۳: تصویر نوبت‌دهی



شکل ۳۴: تصویر پنل پزشک - تاریخچه و گزارش اکوکاردیوگرافی

۳-۷-۱ سرویس پایش زنده علائم حیاتی

این سرویس وظیفه برقراری ارتباط بین پنل کاربری بیمار و API علائم حیاتی سمت سرور را بر عهده دارد و فرآیند دریافت و مدیریت داده‌های پایش را کنترل می‌کند. داده‌های جدید علائم حیاتی در بازه‌های زمانی ۲۰۰۰ میلی‌ثانیه (۲ ثانیه) از سرور دریافت شده و برای نمایش در پنل کاربر در اختیار بخش رابط کاربری قرار می‌گیرند. انتخاب این بازه زمانی به دلیل هماهنگی با دوره ارسال داده توسط دستگاه ESP۳۲ انجام شده است تا اطلاعات با کمترین تأخیر ممکن در اختیار کاربر قرار گیرد. این سرویس امکانات اصلی مربوط به مدیریت فرآیند پایش را فراهم می‌کند؛ به گونه‌ای که امکان شروع یک جلسه پایش جدید، پایان‌دادن به جلسه فعال، دریافت و انتشار لحظه‌ای داده‌های حیاتی و در نهایت ذخیره نتایج نهایی جلسه پایش در پروفایل بالینی بیمار را فراهم می‌سازد.



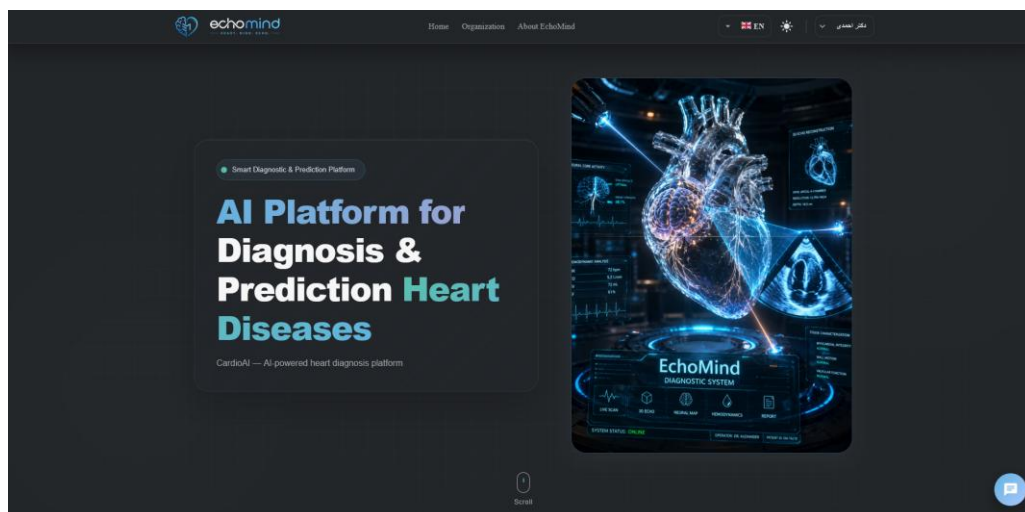
شکل ۳۵: تصویر نمودار زنده پایش ضربان قلب

۳-۷-۲ پشتیبانی چندزبانگی و تم

سرویس مدیریت زبان برای کنترل زبان رابط کاربری استفاده می‌شود و امکان تغییر پویا بین زبان فارسی و انگلیسی را بدون Refresh شدن صفحه فراهم می‌کند. این سرویس همچنین جهت نمایش محتوا را متناسب با زبان انتخابی (راست‌چین یا چپ‌چین) تنظیم می‌کند. سرویس مدیریت پوسته نیز امکان تغییر ظاهر برنامه را فراهم کرده و کاربر می‌تواند بین دو تم تعریف‌شده در سیستم انتخاب کند.



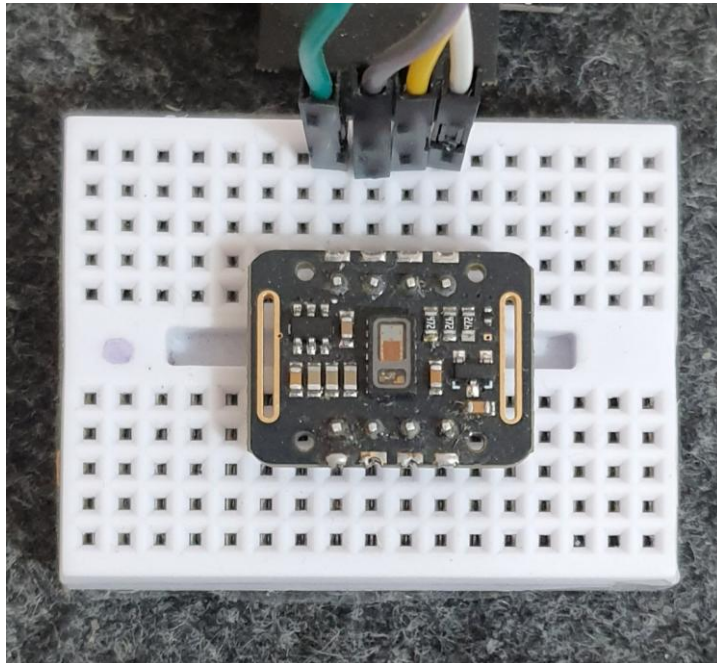
شکل ۳۶: تم روز فرانت



شکل ۳۷: تم شب فرانت

۳-۸ پیاده‌سازی ماژول سخت‌افزاری پایش علائم حیاتی

ماژول سخت‌افزاری پایش زنده علائم حیاتی، بر پایه میکروکنترلر ESP۳۲ و سنسور نوری نبض‌سنجی MAX۳۰۱۰۲ پیاده‌سازی شده و مسئول اندازه‌گیری پیوسته ضربان قلب بیمار است.



شکل ۳۸: سنسور MAX۳۰۱۰۲ (محل قرار گیری انگشت فرد)

۳-۸-۱ ارتباط با سرور

دستگاه پس از اتصال به شبکه Wi-Fi، با استفاده از کتابخانه استاندارد Arduino با نام PubSubClient به بروکر MQTT متصل می‌شود و ارتباط دوطرفه زیر را برقرار می‌سازد: دستگاه در موضوع `devices/ESP۳۲_۰۰۱/cmd` مشترک می‌شود و منتظر دریافت فرمان از سمت Laravel می‌ماند؛ در طول یک جلسه فعال، دستگاه هر دو ثانیه یک‌بار یک پیام JSON در موضوع `devices/ESP۳۲_۰۰۱/data` منتشر می‌کند که شامل شناسه بیمار، شناسه دستگاه، مقدار ضربان اندازه‌گیری‌شده، و یک پرچم بولی اعتبار مقدار است.

۳-۸-۲ منطق ایمنی و کنترل کیفیت داده

- محدودیت مدت زمان جلسه: هر جلسه پایش حداکثر تا سقف مشخص صدوپنجاه ثانیه مجاز است و پس از آن به صورت خودکار خاتمه می یابد.
- توقف خودکار در صورت قطع ارتباط: در صورتی که ارتباط MQTT دستگاه با بروکر بیش از بیست ثانیه قطع بماند، جلسه به صورت خودکار متوقف می شود.
- تشخیص خروج انگشت از سنسور: دستگاه با پایش پیوسته شدت سیگنال نوری مادون قرمز بازتابی، خروج انگشت بیمار را تشخیص می دهد.

۹-۳ جمع‌بندی فصل

در این فصل، جزئیات دقیق فنی پیاده‌سازی هر یک از مؤلفه‌های سیستم شامل بک‌اند **Laravel**، سرویس هوش مصنوعی و چت‌بات، خط پردازش پایتونی اکوکاردیوگرافی، فرانت‌اند **Angular** و ماژول سخت‌افزاری **ESP۳۲** شرح داده شد. ترکیب هدفمند این مؤلفه‌ها، با در نظر گرفتن تصمیمات فنی دقیق در هر لایه، یک سیستم یکپارچه، قابل نگهداری و انعطاف‌پذیر را شکل می‌دهد که هم‌زمان نیازهای اداری روزمره کلینیک و نیازهای پیشرفته تحلیل پزشکی هوشمند و پایش زنده بیمار را پوشش می‌دهد.

فصل چهارم: جمع‌بندی و نتیجه‌گیری

۴-۱ جمع‌بندی نهایی

در این پروژه، یک سامانه یکپارچه کلینیکی طراحی و پیاده‌سازی شد که سه محور اصلی مدیریت اداری کلینیک، تحلیل هوشمند تصویر پزشکی و پایش زنده علائم حیاتی را در قالب یک معماری میکروسرویسی منسجم و مستند گرد هم می‌آورد. تجربه پیاده‌سازی این پروژه نشان داد که به‌کارگیری آگاهانه و هدفمند فناوری‌های متفاوت متناسب با ماهیت هر مسئله، به‌جای اصرار بر یک راه‌حل فناورانه واحد برای همه بخش‌ها، منجر به سیستمی کارآمدتر و قابل‌نگهداری‌تر می‌شود. استفاده هم‌زمان از پایگاه‌داده رابطه‌ای MySQL برای داده‌های ساخت‌یافته اداری و پایگاه‌داده سندگرا MongoDB برای داده‌های پزشکی با ساختار متغیر، امکان مدیریت مناسب و کارآمد هر دو دسته داده را بدون سازش در طراحی هرکدام فراهم کرد. به‌کارگیری مدل‌های یادگیری عمیق برای پردازش تصویر پزشکی، مدل‌های یادگیری ماشین کلاسیک برای برآورد سریع و تفسیرپذیر ریسک، منطق فازی برای تجمیع معقول نتایج نامطمئن اندازه‌گیری، و مدل‌های زبانی بزرگ برای تعامل طبیعی و منعطف با کاربر نهایی، نشان‌دهنده به‌کارگیری ترکیبی و آگاهانه از طیف گسترده‌ای از فناوری‌های هوش مصنوعی است. همچنین طراحی و ادغام ماژول سخت‌افزاری اینترنت اشیا از طریق پروتکل سبک و بلادرنگ MQTT، امکان پایش زنده بیمار را بدون افزودن بار پردازشی یا تأخیر قابل‌توجه به سیستم اصلی فراهم کرد.

از منظر نتیجه‌گیری کلی، این پروژه نشان می‌دهد که ترکیب یک سامانه مدیریت کلینیک با ابزارهای تخصصی هوش مصنوعی می‌تواند بخش قابل‌توجهی از فرآیندهای دستی، تکراری و زمان‌بر تشخیص اولیه بیماری‌های قلبی-عروقی را کاهش دهد و در عین حال، با افزودن قابلیت‌هایی مانند دستیار گفتگو محور و پایش زنده، تجربه کاربری به‌مراتب غنی‌تر و در دسترس‌تری برای بیمار و پزشک فراهم کند. با این حال، لازم به تأکید است که این سیستم در وضعیت فعلی به‌عنوان یک ابزار کمکی و پشتیبان تصمیم‌گیری طراحی شده و نه جایگزینی برای قضاوت بالینی متخصص؛ استفاده نهایی از هرگونه خروجی این سیستم در تصمیم‌گیری‌های درمانی واقعی، مستلزم بازبینی و تأیید نهایی توسط پزشک متخصص است.

۴-۲ پیشنهاد‌های ارائه‌شده جهت توسعه برنامه در آینده

بر اساس تجربه پیاده‌سازی و محدودیت‌های شناسایی‌شده در وضعیت فعلی سیستم، پیشنهاد‌های مشخص زیر برای توسعه و بهبود پروژه در پژوهش‌ها و نسخه‌های آتی ارائه می‌شود:

۱. افزایش دقت مدل‌های تصویری از طریق گسترش داده آموزشی: آموزش مجدد یا تنظیم دقیق مدل‌های کلاس‌بندی نما و قطعه‌بندی حفرات روی مجموعه داده‌های بزرگ‌تر و متنوع‌تر.

۲. مهاجرت پایگاه داده Qdrant به حالت اجرای سرور مستقل: با توجه به محدودیت شناخته‌شده حالت فعلی اجرای فایل‌محلی، برای پشتیبانی از استقرار چند نمونه هم‌زمان سرویس FAISS در محیط تولید.

۳. افزودن قابلیت اندازه‌گیری اکسیژن خون: با توجه به این‌که سنسور سخت‌افزاری MAX۳۰۱۰۲ به صورت بومی از این قابلیت پشتیبانی می‌کند اما در فریمور فعلی مورد استفاده قرار نگرفته است.

۴. پیاده‌سازی سامانه اعلان و یادآوری: افزودن قابلیت ارسال اعلان برای یادآوری مصرف به موقع دارو و نوبت‌های پزشکی پیش‌رو.

۵. گسترش و تکمیل پشتیبانی چندسازمانی: انجام آزمون‌های جامع‌تر برای سناریوهای استقرار هم‌زمان چندین کلینیک مستقل.

۶. افزودن رمزنگاری به ارتباط MQTT: رمزنگاری ارتباط بروکر Mosquitto با استفاده از پروتکل TLS.

۷. انجام آزمون بالینی و اعتبارسنجی رسمی پزشکی: اعتبارسنجی نتایج مدل‌های اندازه‌گیری تصویری و برآورد ریسک از طریق مطالعات تطبیقی و آماری با نظر متخصصین قلب و عروق باتجربه.

پیوست‌ها

پیوست الف: متغیرهای محیطی کلیدی

در ادامه فهرست برخی از متغیرهای محیطی کلیدی مورد استفاده در پیکربندی مؤلفه‌های مختلف سیستم آورده شده است:

• ECHO_MONGO_COLLECTION، ECHO_MONGO_DB، ECHO_MONGO_URI: تنظیمات

اتصال پایگاه داده MongoDB برای خط پردازش پایتونی.

• LARAVEL_PUBLIC_RESULT_PATH: مسیر خروجی مشترک فایل سیستمی میان خط پردازش

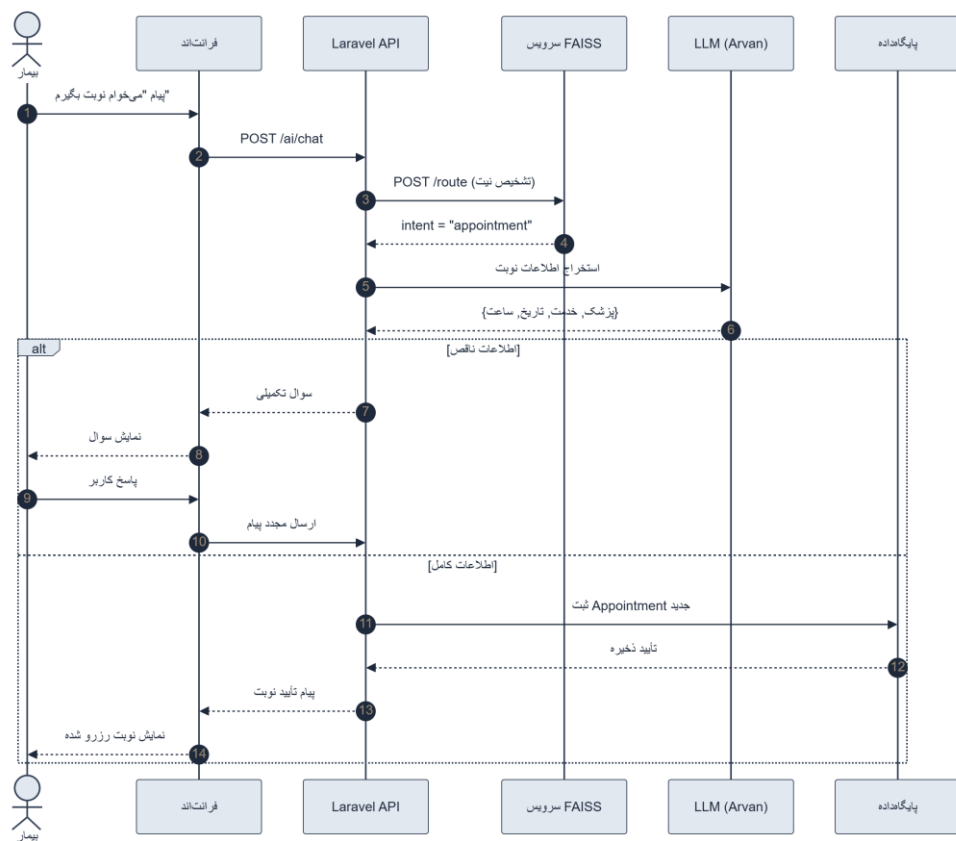
پایتونی و بک‌اند Laravel.

• ARVAN_AI_MODEL، ARVAN_AI_BASE_URL، ARVAN_AI_API_KEY: پیکربندی اتصال

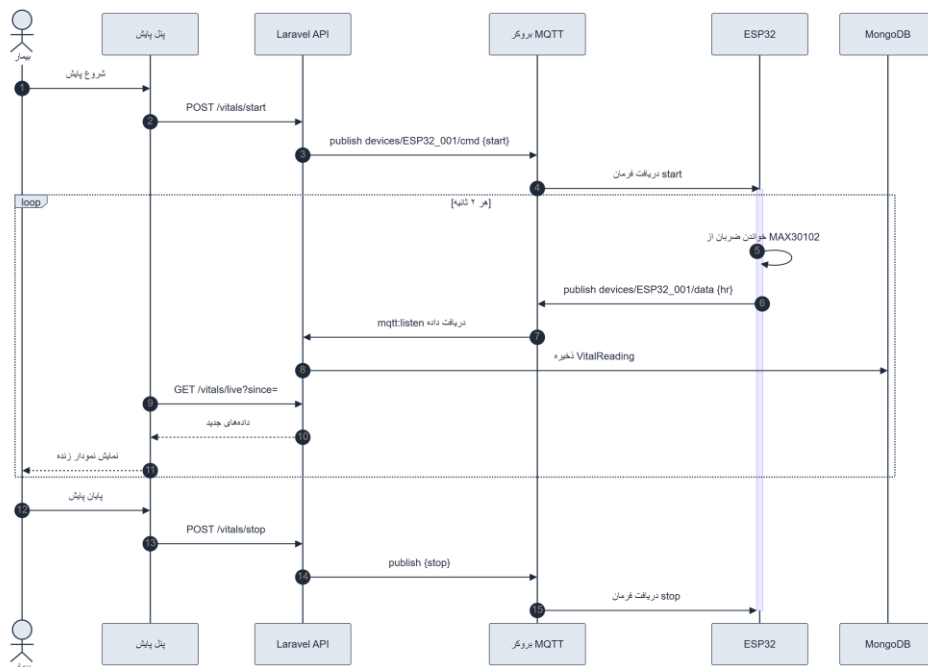
به سرویس مدل زبانی بزرگ آروان.

پیوست ب: نمودارهای تکمیلی

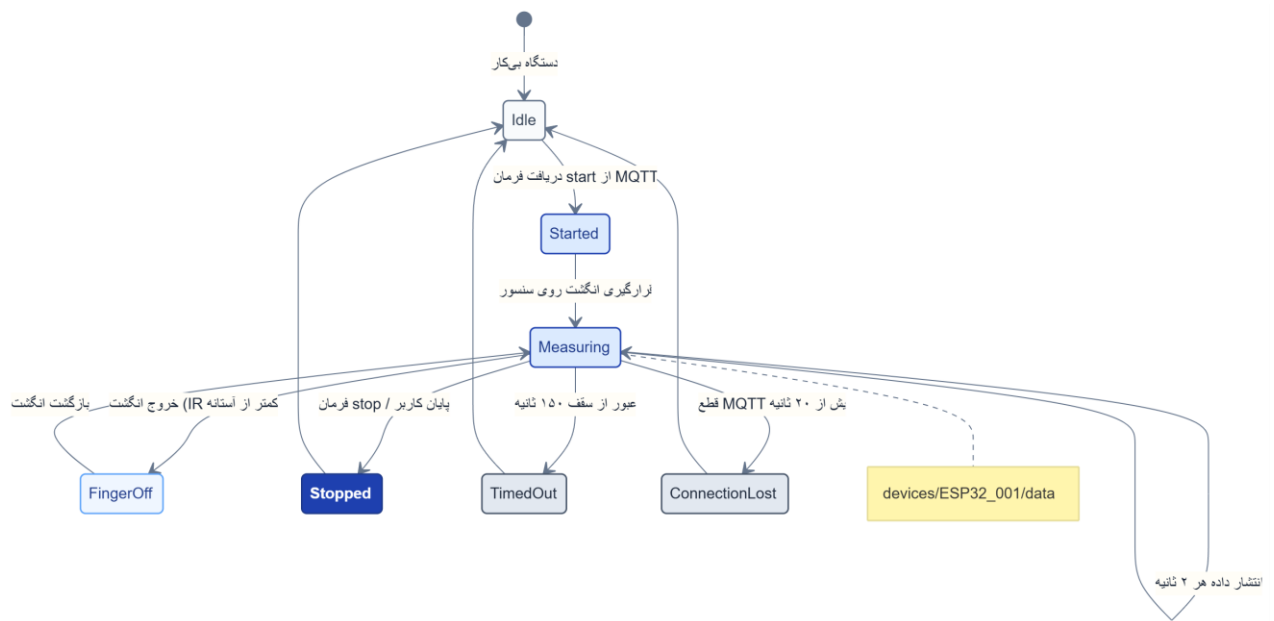
نمودارهای صفحه بعد جهت تکمیل تحلیل و طراحی سیستم، به عنوان پیوست ارائه می شوند.



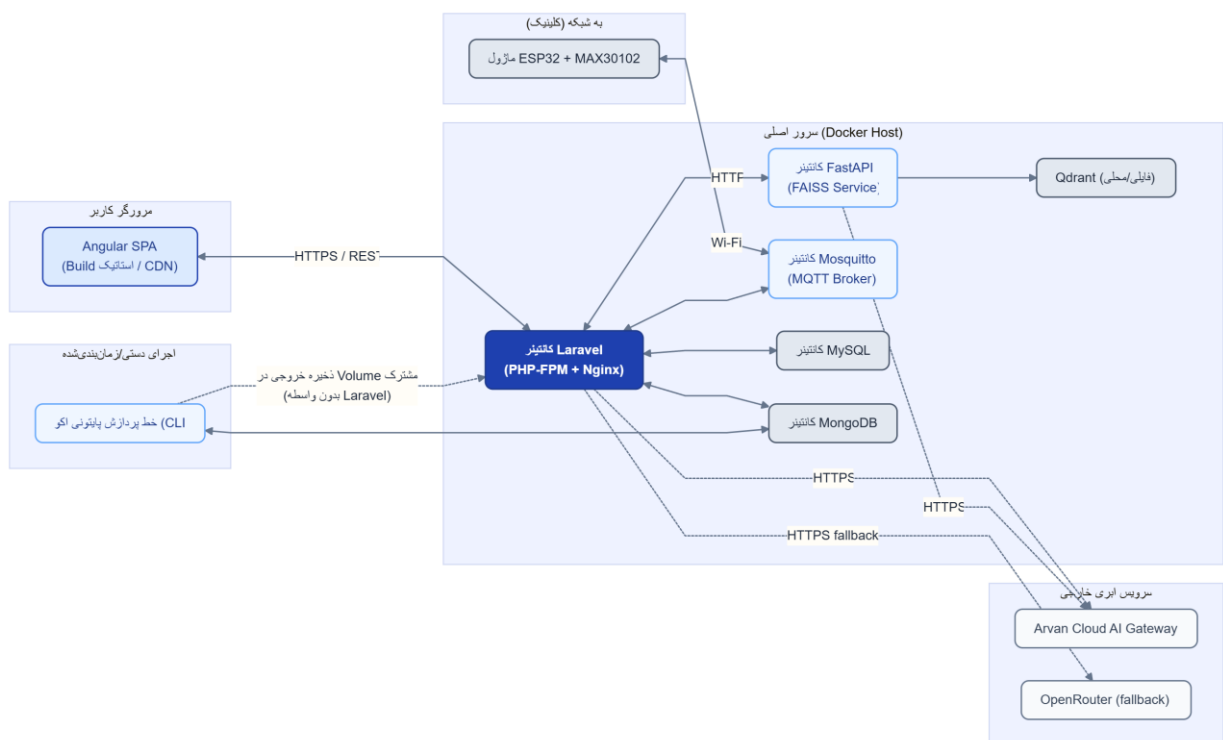
شکل ۳۹: نمودار توالی - فرآیند نوبت‌دهی



شکل ۴۰: نمودار توالی - پایش علائم حیاتی



شکل ۴۱: نمودار حالت - جلسه پایش علائم حیاتی



شکل ۴۲: نمودار استقرار سیستم

پیوست ج: کدهای تکمیلی

کد یک: این بخش پل ارتباطی بین فرانت و سند MongoDB بیمار است (همان سندی که پایپ‌لاین پایتونی در نهایت می‌نویسد). دو نکته‌ی این سرویس قابل تأکید است: تشخیص هوشمند شناسه‌ی بیمار (چون در مونگو گاهی رشته و گاهی عدد ذخیره شده) و ساخت خودکار URL دانلود برای هر مسیر فایل که در سند پیدا می‌شود. `normalizeFileAddress` مسیر فایل را با `realpath` به مسیر مطلق تبدیل و بررسی می‌کند که همچنان داخل پوشه‌ی `public/echos` باشد؛ این جلوی خواندن فایل‌های خارج از پوشه‌ی مجاز را می‌گیرد، حتی اگر داده‌ی مونگو (که منبعش خروجی پایپ‌لاین پایتونی است) دستکاری شده یا نامعتبر باشد.

```
1 private function findPatientDocument(string $patientId)
2 {
3     $candidates = [$patientId];
4     if (ctype_digit($patientId)) {
5         $candidates[] = (int) $patientId;
6     }
7
8     foreach (['_id', 'patient_info.id', 'patient_id'] as $field) {
9         foreach ($candidates as $candidate) {
10             $document = $this->findOne([$field => $candidate]);
11             if ($document) {
12                 return $document;
13             }
14         }
15     }
16
17     return null;
18 }
```

```
1 private function appendFileLinks($value, ?string $parentKey = null)
2 {
3     if (!is_array($value)) {
4         return $value;
5     }
6
7     foreach ($value as $key => $item) {
8         if (is_array($item)) {
9             $value[$key] = $this->appendFileLinks($item, is_string($key) ? $key : $parentKey);
10            continue;
11        }
12
13        if (!is_string($item) || !$this->looksLikePublicFilePath($item)) {
14            continue;
15        }
16
17        $address = $this->normalizeFileAddress($item);
18        if (!$address) {
19            continue;
20        }
21
22        if (is_string($key)) {
23            $value[$key . '_address'] = $address;
24            $value[$key . '_url'] = $this->fileUrl($address);
25            $value[$key . '_exists'] = $this->publicFileExists($address);
26        }
27    }
28
29    if ($this->isStringList($value) && $this->looksLikeFileList($parentKey, $value)) {
30        return [
31            'items' => array_values($value),
32            'links' => array_values(array_filter(array_map(function ($path) {
33                $address = $this->normalizeFileAddress($path);
34
35                if (!$address) {
36                    return null;
37                }
38
39                return [
40                    'path' => $path,
41                    'address' => $address,
42                    'url' => $this->fileUrl($address),
43                    'exists' => $this->publicFileExists($address),
44                ];
45            }, $value))),
46        ];
47    }
48
49    return $value;
50 }
```

کد دو: تابع `route_request` نیت پیام را با ترکیب دو روش تعیین می‌کند: تطبیق کلمات کلیدی صریح (مثل «آدرس» یا «رزرو کن») و جست‌وجوی برداری در Qdrant با امبدینگ محلی (Sentence-Transformer). اولویت با کلمات کلیدی صریح است؛ در غیاب آن‌ها امتیاز جست‌وجوی برداری تصمیم می‌گیرد و در نهایت، در حالت مبهم، سیستم به نفع «رزرو نوبت» تصمیم می‌گیرد (چون نرخ خطای آن کم‌هزینه‌تر است).

```
1 @app.post("/route")
2 def route_request(request: UserRequest):
3     try:
4         user_text = request.sentence
5
6         has_support_keyword = any(keyword in user_text for keyword in SUPPORT_KEYWORDS)
7         has_appointment_keyword = any(keyword in user_text for keyword in APPOINTMENT_KEYWORDS)
8
9         query_vector = encode_text(request.sentence)
10        search_result = client.query_points(
11            collection_name=collection_name,
12            query=query_vector,
13            limit=5
14        )
15
16        if not search_result.points:
17            intent = "support" if has_support_keyword else "appointment"
18            return {"intent": intent, "score": 0.0, "reason": "no vector results"}
19
20        top_score = search_result.points[0].score
21        top_category = search_result.points[0].payload.get("category", "unknown")
22
23        if has_support_keyword and not has_appointment_keyword:
24            intent, reason = "support", "explicit support keyword"
25        elif has_appointment_keyword and not has_support_keyword:
26            intent, reason = "appointment", "explicit appointment keyword"
27
28        else:
29            THRESHOLD = 0.45
30            if top_score >= THRESHOLD:
31                intent = top_category
32                reason = "high vector score"
33            else:
34                intent = "appointment"
35                reason = "low score fallback"
36
37        print(f"Final Decision → Intent: {intent} (Score: {top_score:.3f}, Reason: {reason})")
38
39        return {
40            "intent": intent,
41            "score": top_score,
42            "matched_sentence": search_result.points[0].payload.get("sentence_text"),
43            "category": top_category,
44            "reason": reason
45        }
46
47    except Exception as e:
48        raise HTTPException(status_code=500, detail=str(e))
```

کد سه: تابع main پنج گام اصلی بخش فرآیند اکاردیوگرافی را به ترتیب طی می کند: خواندن اطلاعات بیمار (سن، جنسیت، قد، وزن، بیماری های زمینه ای) از مونگو، برآورد اولیه از ریسک با اجرای دو مدل ML با استفاده از اطلاعات بیمار، پردازش تک تک ویدیوهای بیمار و استخراج اندازه های بالینی، جمعیت فازی نتایج به دست آمده از اکو، و در نهایت تولید گزارش نهایی با کمک مدل زبانی

```
1 def main() → None:
2     args = build_parser().parse_args()
3
4     output_root = ensure_dir(RESULT_DIR)
5     visit_date = datetime.now().strftime("%Y-%m-%d")
6
7
8     input_path = Path(args.video).expanduser().resolve()
9     patient_id = input_path.name
10
11     patient_config = get_patient_config(patient_id)
12     print(f"patient data recive from mongo: {patient_config.get('name', 'بدون نام')}")
13
14     ml_result = run_ml_analysis(patient_config)
15
16     if not input_path.is_dir():
17         print(f"خطا: {input_path} مسیر باید یک پوشه باشد")
18         return
19
20     video_extensions = {".avi", ".mp4", ".mov", ".mkv", ".wmv"}
21     video_files = [f for f in input_path.iterdir() if f.suffix.lower() in video_extensions]
22     if not video_files:
23         print(f"نیافت شد {input_path} فایل ویدیو در")
24         return
25
26     all_rows: list[dict] = []
27     for video_path in video_files:
28         print(f"\n پردازش: {video_path.name}")
29         try:
30             rows = process_video(video_path, output_root, patient_config=patient_config)
31             all_rows.extend(rows)
32         except Exception as exc:
33             print(f"خطا در {video_path.name}: {exc}")
34             all_rows.append({"video_name": video_path.name, "video_path": str(video_path),
35                             "patient_id": patient_id, "error": str(exc)})
36
37     fuzzy_result = aggregate_and_evaluate_fuzzy(
38         output_root, patient_id, visit_date, patient_config,
39         rows=all_rows,
40     )
41
42     generate_and_save_final_report(
43         output_root=output_root, patient_id=patient_id, visit_date=visit_date,
44         patient_config=patient_config, ml_result=ml_result,
45         fuzzy_result=fuzzy_result, all_rows=all_rows,
46     )
47
48
```

کد چهار: این تابع خروجی نهایی زیرسیستم یادگیری ماشین پروژه است. پس از بررسی ورودی‌ها و محاسبه‌ی فیچرهای ترکیبی، خروجی دو مدل با میانگین وزنی ترکیب می‌شوند؛ وزن هر مدل «ضریب اطمینان» (confidence) آن است عددی که نشان می‌دهد چقدر از فیلدهای موردنیاز آن مدل برای این بیمار مشخص در دسترس بوده‌اند. یعنی اگر داده‌ی ورودی برای یکی از دو مدل ناقص‌تر باشد، آن مدل اثر کمتری در نتیجه‌ی نهایی می‌گذارد. احتمال ترکیبی در نهایت به سه سطح شدت گسسته تبدیل می‌شود: پایین، متوسط، بالا. خروجی نهایی علاوه بر میزان ریسک، شامل فهرست عوامل خطر شناسایی شده نیز می‌باشد (برای فیچرهایی که از سطح نرمال خارج بوده‌اند) تا اطلاعات کامل‌تری در اختیار مدل زبانی برای تولید گزارش نهایی قرار بگیرد.

```

1
2 def predict(self, patient_data: dict[str, Any]) → dict[str, Any]:
3
4     hd = self._predict_hd(patient_data)
5     cv = self._predict_cv(patient_data)
6
7     hd_conf, cv_conf = hd.get("confidence", 0.0), cv.get("confidence", 0.0)
8     total_conf = hd_conf + cv_conf
9     combined_prob = (
10         (hd["probability"] * hd_conf + cv["probability"] * cv_conf) / total_conf
11         if total_conf > 0 else
12         (hd["probability"] + cv["probability"]) / 2
13     )
14
15     if combined_prob ≥ 0.70: severity = "HIGH"
16     elif combined_prob ≥ 0.45: severity = "MODERATE"
17     else: severity = "LOW"
18
19     risk_factors = self._detect_risk_factors(hd.get("_engineered", {}), cv.get("_engineered", {}))
20     bmi_val = cv.get("_engineered", {}).get("bmi")
21
22     for r in (hd, cv):
23         r.pop("_engineered", None)
24
25     return {
26         "hd_result": hd,
27         "cv_result": cv,
28         "combined_score": round(combined_prob * 100, 1), # ترکیب سه
29         "combined_prob": round(combined_prob, 4),
30         "severity": severity,
31         "risk_factors": risk_factors,
32         "bmi": round(bmi_val, 1) if bmi_val else None,
33         "confidence": round((hd_conf + cv_conf) / 2, 3),
34     }
35

```

کد پنج: این بخش، اولین گام پردازش تصویر روی هر فریم اکو در مرحله‌ی محاسبه‌ی اندازه‌ی دهلیزها است و هدفش پیدا کردن مرکز تشریحی قلب به صورت خودکار است نقطه‌ای که تمام محاسبات بعدی (اندازه‌گیری حفره‌ها، جداسازی دهلیزها) نسبت به آن انجام می‌شود. روش کار مبتنی بر یک ویژگی شناخته‌شده در تصاویر اکو است: دیواره‌های بافتی قلب در تصویر روشن‌تر از فضای پر از خون داخل حفره‌ها دیده می‌شوند. برای سنجش این روشنایی در یک جهت مشخص، از یک نقطه‌ی کاندید در بازه‌ای از زوایا چند پرتو مستقیم فرستاده می‌شود؛ روی مسیر هر پرتو، پیکسل‌ها نمونه‌برداری و میانگین روشنایی‌شان محاسبه می‌شود، و در نهایت پرتویی که بیشترین میانگین روشنایی را در آن جهت داشته باشد به عنوان بهترین پرتوی آن جهت انتخاب می‌شود. این سنجش برای یافتن مرکز واقعی قلب در قالب یک جست‌وجوی شبکه‌ای (grid search) روی نقاط کاندید تکرار می‌شود. محدوده‌ی جست‌وجو به بخشی از تصویر محدود شده که آناتومی این نمای اکو معمولاً قلب را در آن قرار می‌دهد. از هر نقطه‌ی کاندید، چهار پرتو به چهار جهت اصلی (راست، پایین، چپ، بالا) فرستاده و بهترین پرتوی هر جهت طبق روش بالا پیدا می‌شود؛ مجموع روشنایی این چهار پرتو، امتیاز آن نقطه‌ی کاندید را می‌سازد. نقطه‌ای که این مجموع در آن بیشینه باشد، به عنوان مرکز قلب انتخاب می‌شود چون چنین نقطه‌ای بیشترین هم‌پوشانی را با دیواره‌های اطراف خود دارد. همراه با این مرکز، چهار نقطه‌ی انتهایی پرتوهای برنده نیز نگه داشته می‌شوند. اهمیت این مرحله در خط پردازش: مرکز و پرتوهای منتخب مبنای تمام مراحل بعدی هستند از آن‌ها برای پیدا کردن نقطه‌ی شروع (seed) پر کردن هر حفره، تفکیک دهلیز چپ از راست، و در نهایت محاسبه‌ی مساحت هر حفره استفاده می‌شود.

```

1 def best_ray(cx: int, cy: int, angles: range, length: int) -> tuple[float, tuple[int, int]]:
2     maxv, best_end = -1.0, (cx, cy)
3     for a in angles:
4         rad = math.radians(a)
5         dx, dy = length * math.cos(rad), length * math.sin(rad)
6         xs = np.linspace(cx, cx + dx, int(length)).astype(int)
7         ys = np.linspace(cy, cy + dy, int(length)).astype(int)
8         valid = (xs >= 0) & (xs < w) & (ys >= 0) & (ys < h)
9         if not np.any(valid):
10             continue
11         val = float(np.mean(blur_rays[ys[valid], xs[valid]]))
12         if val > maxv:
13             maxv = val
14             best_end = (int(cx + dx), int(cy + dy))
15     return maxv, best_end
16
17 ray_len = int(h * 0.33)
18 best_center = (w // 2, h // 2)
19 best_score = -1.0
20 best_rays: list[tuple[int, int]] = []
21
22 for cy in range(int(h * 0.4), int(h * 0.7), 5):
23     for cx in range(int(w * 0.4), int(w * 0.6), 5):
24         s_r, e_r = best_ray(cx, cy, range(-30, 31, 2), ray_len)
25         s_d, e_d = best_ray(cx, cy, range(60, 121, 2), ray_len)
26         s_l, e_l = best_ray(cx, cy, range(150, 211, 2), ray_len)
27         s_u, e_u = best_ray(cx, cy, range(240, 301, 2), ray_len)
28         tot = s_r + s_d + s_l + s_u
29         if tot > best_score:
30             best_score = tot
31             best_center = (cx, cy)
32             best_rays = [e_r, e_d, e_l, e_u]
33

```

کد شش: این بخش از کد، دیتای پزشکی است که توسط متخصص قلب (کاردیولوژیست) بر اساس استانداردهای مرجع اکوکاردیوگرافی (نظیر دستورالعمل‌های ASE) تعیین شده یعنی این اعداد نتیجه‌ی محاسبه یا یادگیری ماشین نیستند، بلکه ورودی دانش خبره (expert knowledge) به سیستم هستند که به شکل یک جدول قابل تنظیم در کد نگه داشته شده‌اند. هر پارامتر بالینی (ضخامت دیواره، قطر بطن، حجم دهلیز و...) یک بازه‌ی مرجع چهارتایی دارد: حد پایین طبیعی، حد بالای طبیعی، حد خفیف، و حد شدید. این بازه‌ها جدا برای مرد و زن تعریف شده‌اند، چون آناتومی قلب بین دو جنس تفاوت دارد. نقش این جدول در جریان کار سیستم: خروجی خام پایپ‌لاین پردازش تصویر (مثلاً «ضخامت سپتوم = ۱۳.۸ میلی‌متر») یک عدد صرف است و به‌تنهایی برای گزارش پزشکی قابل فهم نیست. این جدول همان مرجعی است که موتور منطق فازی پروژه با آن مقایسه انجام می‌دهد و آن عدد خام را به یک برجسب بالینی معنادار تبدیل می‌کند: طبیعی، خفیف، یا شدید. به همین دلیل خروجی این مرحله «فازی» نامیده می‌شود و به‌جای یک تصمیم قطعی دوحالتی (سالم/بیمار)، یک طیف درجه‌بندی شده از شدت تولید می‌شود که به شرایط واقعی بالینی نزدیک‌تر است. نتیجه‌ی نهایی این مقایسه‌ها به همراه دلایل هر برجسب، مستقیماً در بخش «یافته‌های اکو» گزارش نهایی بیمار درج می‌شود.

```

1 PARAM_CONFIG = {
2     # --- انورت
3     'aortic_root': {'male': (20, 37, 42, 50), 'female': (20, 33, 38, 45), 'needs_bsa': False},
4     'aortic_asc': {'male': (22, 38, 43, 55), 'female': (20, 35, 40, 50), 'needs_bsa': False},
5     # انورت صعودی
6
7     # --- دهلیزها
8     'la_volume': {'male': (16, 34, 41, 50), 'female': (16, 34, 41, 50), 'needs_bsa': True},
9     # حجم دهلیز چپ
10    'ra_volume': {'male': (25, 39, 46, 60), 'female': (21, 33, 40, 55), 'needs_bsa': True},
11    # حجم دهلیز راست
12
13    # --- بطن راست
14    'rv_diameter': {'male': (20, 41, 46, 55), 'female': (20, 41, 46, 55), 'needs_bsa': False},
15    # قطر بطن راست
16    'rv_wall': {'male': (1, 5, 7, 10), 'female': (1, 5, 7, 10), 'needs_bsa': False},
17    # ضخامت دیواره بطن راست (mm): 5 ≥ Mild ≤7, Severe >7
18
19    # --- بطن چپ
20    'lv_edv': {'male': (35, 74, 89, 110), 'female': (35, 61, 73, 90), 'needs_bsa': True},
21    # حجم پایان دیاستولی بطن چپ
22    'lv_esv': {'male': (15, 31, 39, 55), 'female': (15, 24, 32, 45), 'needs_bsa': True},
23    # حجم پایان سیستولی بطن چپ
24
25    # --- ضخامت دیواره‌ها
26    'ivs_thickness': {'male': (6, 10, 13, 17), 'female': (6, 9, 12, 16), 'needs_bsa': False},
27    # ضخامت سپتوم بین بطنی
28    'pw_thickness': {'male': (6, 10, 13, 17), 'female': (6, 9, 12, 16), 'needs_bsa': False},
29    # ضخامت دیواره خلفی
30
31    # --- قطر بطن چپ
32    'lv_diameter': {'male': (42, 58, 64, 75), 'female': (38, 52, 58, 65), 'needs_bsa': False},
33    # قطر داخلی بطن چپ
34
35    # --- شریان رویی
36    'pa_diameter': {'male': (15, 25, 30, 40), 'female': (15, 25, 30, 40), 'needs_bsa': False},
37    # قطر شریان رویی
38 }

```

کد هفت: این کد نقشه‌ی کلی دسترسی به بخش‌های مختلف سامانه را تعریف می‌کند و سه سطح کاربری اصلی پروژه را از هم جدا می‌سازد:

بخش عمومی: صفحات معرفی و ورود، بدون نیاز به احراز هویت، برای هر بازدیدکننده در دسترس است. پنل مدیریت: مخصوص نقش ادمین سامانه.

پنل پزشک: مخصوص کادر درمان، برای مشاهده و پیگیری وضعیت بیماران.

هر بخش محافظت‌شده پیش از نمایش، از یک لایه‌ی کنترل دسترسی عبور می‌کند که بررسی می‌کند کاربر لاگین کرده و نقش او متناسب با آن بخش است یا نه؛ در غیر این صورت کاربر یا به صفحه‌ی ورود یا به صفحه‌ی خطای دسترسی هدایت می‌شود. این ساختار تضمین می‌کند که هر نقش فقط به بخشی از سامانه دسترسی دارد که برایش تعریف شده (مثلاً بیمار عادی امکان مشاهده‌ی پنل پزشک یا پنل مدیریتی را ندارد). تشخیص لاگین‌بودن و نقش کاربر بر اساس همان توکنی انجام می‌شود که پس از ورود موفق از سمت بک‌اند دریافت و در `localStorage` مرورگر ذخیره می‌شود؛ این لایه‌ی کنترل دسترسی در فرانت، همان توکن ذخیره‌شده و اطلاعات نقش داخل آن را می‌خواند و بر همان اساس تصمیم می‌گیرد که اجازه‌ی ورود به مسیر درخواستی داده شود یا خیر.

```
1 export const routes: Routes = [
2   {
3     path: '',
4     loadChildren: () => import('./index/index.routes').then((m) => m.routes),
5   },
6   {
7     path: 'panel',
8     loadChildren: () => import('./panel/panel.routes').then((m) => m.routes),
9     canActivate: [authGuard],
10    data: { roles: ['admin', 'doctor'] },
11  },
12
13  {
14    path: 'dr-panel',
15    loadChildren: () => import('./doctor-panel/doctor-panel.routes').then((m) => m.routes),
16    canActivate: [authGuard],
17    data: { roles: ['doctor'] },
18  },
19  {
20    path: 'auth',
21    loadChildren: () => import('./auth/auth.routes').then((m) => m.routes),
22  },
23  {
24    path: '**',
25    redirectTo: '',
26  },
27 ];
```

فهرست منابع

منابع فارسی:

در این پروژه از منبع مستقل فارسی خاصی استناد مستقیم صورت نگرفته است.

منابع لاتین:

- [۱] Laravel Documentation, Taylor Otwell, <https://laravel.com/docs>, ۲۰۲۴.
- [۲] Angular Documentation, Google, <https://angular.dev>, ۲۰۲۴.
- [۳] FastAPI Documentation, Sebastian Ramirez, <https://fastapi.tiangolo.com>, ۲۰۲۴.
- [۴] Zadeh, L. A., "Fuzzy Sets", Information and Control, Elsevier, vol. ۸, no. ۳, pp. ۳۳۸-۳۵۳, ۱۹۶۵.
- [۵] MQTT Version ۵.۰ Specification, OASIS Standard, <https://mqtt.org>, ۲۰۱۹.
- [۶] EchoNet-Dynamic Dataset, Kaggle, <https://www.kaggle.com/datasets/mahnurrahman/echonet-dynamic>.
- [۷] Heart Disease Dataset, Kaggle, <https://www.kaggle.com/datasets/mahmoudshaheen1134/heart-disease-data>.
- [۸] Cardiovascular Dataset, Kaggle, <https://www.kaggle.com/datasets/ritunandhan/cardiovascular-dataset>.
- [۹] EchoNet Measurements, GitHub Repository, <https://github.com/echonet/measurements/tree/main/Measurement>.
- [۱۰] Echo View Classifier, GitHub Repository, <https://github.com/raventan90/echo-view-classifier>



echomind

— HEART. MIND. ECHO. —

Abstract

Cardiovascular diseases remain among the leading causes of death worldwide, and their early diagnosis through non-invasive imaging techniques such as echocardiography plays a vital role in reducing long-term complications. However, manual interpretation of echocardiographic images is time-consuming and prone to inter-observer variability, while specialized cardiac clinics also require an integrated software infrastructure for managing appointments, staff, and patient communication. This project presents Echo, a system designed to address both needs simultaneously. Echo is an end-to-end clinical platform built on a lightweight microservices architecture comprising five independent components: a Laravel-based backend responsible for administrative operations and multi-role authentication; an AI service built on FastAPI and the Qdrant vector database, providing intent routing and retrieval-augmented generation for a conversational assistant; a Python processing pipeline that automatically analyzes echocardiographic videos, including view classification, cardiac cycle event detection, chamber measurement, and volume estimation, combined with deep learning, fuzzy logic, and classical machine learning models for cardiovascular risk estimation; an Angular-based frontend serving multi-role user panels; and an ESP32-based hardware module for real-time heart-rate monitoring over the MQTT protocol. The results of this project demonstrate that a deliberate combination of heterogeneous technologies, each matched to the nature of its sub-problem, enables the construction of an integrated, scalable, and efficient system that addresses both the administrative needs of a clinic and the advanced requirements of intelligent medical analysis.

Keywords: Echocardiography, Deep Learning, Fuzzy Logic, Internet of Things, Clinic Management, Conversational Assistant, Machine Learning.



Zanjan University
Faculty of Engineering
Department of Computer Engineering

Bachelor's Thesis

Title:

An Intelligent Echocardiography Process Management and Analysis System
Using Image Processing, Fuzzy Inference, and Heart Rate Monitoring Hardware

Supervisor:

Dr. Davood Mohammadpour Zanjani

Authors:

Narges Ahangar

Student ID: 401463120

Mohammadreza Mehri

Student ID: 401463176

July 2026